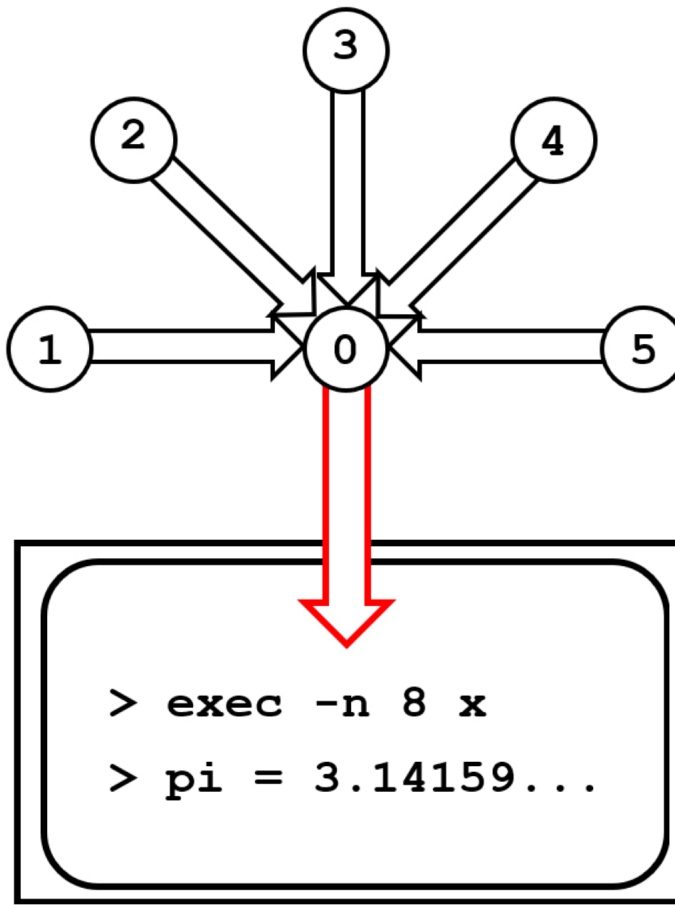
Валентин Юльевич Арьков

Организация параллельных процессов

Учебное пособие



Валентин Юльевич Арьков

Организация параллельных процессов

Учебное пособие

Издательские решения
По лицензии Ridero
2020

УДК 004
ББК 32.973
А89

Шрифты предоставлены компанией «ПараТайп»

Арьков Валентин Юльевич

А89 Организация параллельных процессов : Учебное пособие / Валентин Юльевич Арьков. — [б. м.] : Издательские решения, 2020. — 88 с.
ISBN 978-5-4498-6204-4

В данной работе мы познакомимся с организацией высокопроизводительных вычислений на уровне параллельных процессов. При этом большую задачу приходится разделить на несколько частей, а выполнение каждой части работы поручить отдельному процессу.

Первые шаги мы рассмотрим подробно, а потом отпустим читателя в свободное плавание.

УДК 004
ББК 32.973

12+ В соответствии с ФЗ от 29.12.2010 №436-ФЗ

ISBN 978-5-4498-6204-4 © Валентин Юльевич Арьков, 2020

ВВЕДЕНИЕ

В данной работе мы познакомимся с организацией высокопроизводительных вычислений на уровне параллельных процессов. При этом большую задачу приходится разделить на несколько частей, а выполнение каждой части работы поручить отдельному процессу.

Это будет самое первое знакомство на очень простых примерах.

Задание. Прочитайте в Википедии статью «Процесс (информатика)» и выясните, что такое ПРОЦЕСС и как он связан с ПРОГРАММОЙ.

Параллельные процессы могут выполняться на одном локальном персональном компьютере, или на другом (удалённом) персональном компьютере, или на нескольких отдельных компьютерах, или на большом вычислительном кластере, который состоит из некоторого количества самостоятельных материнских плат, объединённых скоростной локальной сетью.

Задание. Прочитайте в Википедии статью «Кластер (группа компьютеров)» и найдите определение КЛАСТЕРА и виды кластеров.

0. ОРГАНИЗАЦИЯ РАБОТЫ

Для серьёзного изучения предмета параллельных вычислений можно найти несколько книг на русском языке [1 — 9]. Перед этим имеет смысл получить общее представление об изучаемой технологии. Разобрать какой-нибудь сквозной пример с пошаговым руководством. На западе это называется TUTORIAL. Мы добавили к такому практическому подходу простенькие задания для расширения кругозора и общего понимания материала.

Данная работа продолжает серию работ по параллельному программированию [10, 11].

В работе [10] мы познакомились со стандартными методами работы с потоками в операционной системе **Microsoft Windows**.

Далее в работе [11] мы освоили основы работы с параллельными потоками по технологии **OpenMP**. Теперь пришло время для параллельных процессов по технологии **MPI**. Оформление отчёта мы подробно описали в работе [12]. Анализ данных с помощью сводных таблиц мы подробно рассмотрели в работе [13].

Задание. Скачайте пособия [10 — 13] по ссылкам в списке литературы.

0.1. ОТЧЁТ

Отчёт будем оформлять в виде многостраничного файла **Microsoft Excel.** Напомним, что студентам нашего вуза предоставляется лицензия на установку **Microsoft Office 36**5. Подробности на сайте:

https://www.ugatu.su/

Желающие могут использовать любой другой пакет типа электронных таблиц, например, **Libre Office, Таблицы Гугл** или что угодно ещё.

Страницы отчёта нумеруем следующим образом: **01**, **02**, **03** и так далее. Номер страницы указываем на вкладке очередного листа внизу окна **Excel**.

Первая страница отчёта — титульный лист со всеми необходимыми сведениями.

Вторая страница отчёта — оглавление со ссылками на все страницы документа.

Далее идёт сам отчёт. Все подробности и инструкции — в работе [12].

Задание. Создайте файл отчёта, заполните титульный лист и сохраните под коротким информативным названием.

0.2. ВАРИАНТЫ ЗАДАНИЙ

Во всех работах мы используем одно и то же задание — параллельное численное интегрирование. Номер варианта задания — последняя цифра номера зачётки. Если последняя цифра ноль, берём вариант **10**. Нулевой вариант используется только для демонстрации технологии и разбора сквозного примера.

Задание. Определите номер своего варианта и укажите его на титульном листе отчёта.

Исходные данные для численного интегрирования мы используем одни и те же во всей серии работ. Это нелинейная функция с индивидуальным набором коэффициентов и пределы интегрирования. Таблица с вариантами заданий есть в предыдущем пособии [11].

Задание. Опишите свой вариант задания в отчёте на странице 03, включая точное аналитическое решение задачи.

0.3. СРЕДА РАЗРАБОТКИ

В данной работе мы используем среду разработки **Visual Studio Community Edition.** Она бесплатна для учебных и некоммерческих целей. Мы будем использовать компилятор языка **Си**. Процесс установки и компиляции мы описали в работе [10].

Желающие могут использовать любой другой компилятор языка **Си**. Главное, чтобы компилятор поддерживал технологию **MPI**.

Задание. Запустите среду разработки и убедитесь в её работоспособности.

1. ТЕХНОЛОГИЯ MPI

В предыдущей работе мы рассмотрели технологию **OpenMP**. В данной работе нам предстоит познакомиться с одной из наиболее распространённых технологий распараллеливания под названием **MPI**.

Задание. Прочитайте в Википедии начало статьи «Message Passing Interface» и выясните, на какие платформы ориентированы технологии **MPI** и **OpenMP**.

Можно сказать, что MPI — это только общий стандарт. Для написания программы нам потребуются соответствующие средства программирования (библиотека). Это будет называться РЕАЛИЗАЦИЯ MPI.

Задание. Прочитайте в Википедии окончание статьи «Message Passing Interface» и выясните, сколько реализаций MPI существует и для каких операционных систем.

Для организации совместной работы потребуется наладить общение между вычислительными процессами. В технологии MPI это коммуникация в форме ОБМЕНА СООБЩЕНИЯМИ.

Задание. Прочитайте в Википедии статью «Межпроцессное взаимодействие» и выясните, какие механизмы взаимодействия процессов может обеспечить операционная система.

2. УСТАНОВКА БИБЛИОТЕКИ MPI

В данной работе мы рассмотрим работу в среде **Microsoft Visual Studio Community Edition** — бесплатной для учебных и некоммерческих целей. В предыдущих работах мы рассмотрели процедуру установки **Visual Studio** [10]. Желающие могут использовать любой другой компилятор м поддержкой **MPI**.

Компилятор позволяет организовать вызовы библиотечных функций MPI. Но для этого потребуется установить библиотеку MPI. В данной работе мы рассмотрим работу с библиотекой **MPICH**. Это одна из реализаций стандарта MPI. Желающие могут использовать любую другую библиотеку MPI.

Задание. Прочитайте в Википедии статью «MPICH» и выясните, какие известные компании участвовали в разработке и как расшифровывается название библиотеки.

Процедура установки библиотеки MPI в общих чертах демонстрируется в ролике [15]. Далее мы разберём каждый шаг установки более подробно.

Задание. Посмотрите видеоролик [15].

Нам нужно скачать и установить библиотеку MPICH2. Переходим на сайт проекта:

http://www.mpich.org/

Переходим в раздел **Downloads**.

Находим ссылку на предыдущие выпуски:

Older releases are available here (рис.2.1).

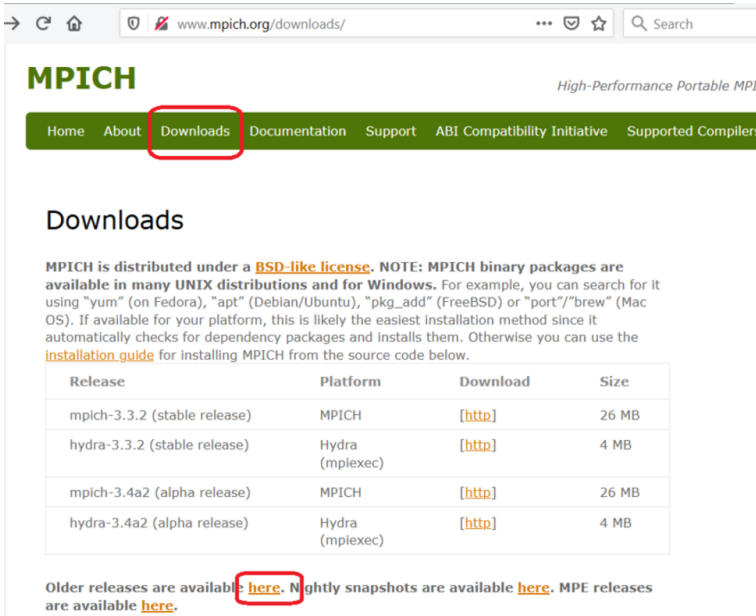


Рис. 2.1. Сайт проекта

Переходим по указанной ссылке и получаем список разных версий (рис. 2.2).



Рис. 2.2. Версии библиотеки

При первом знакомстве с библиотекой начнём с чего-нибудь простого. Последние версии предлагаются в виде исходных текстов. Мы будем искать предыдущие версии с готовой программой установки **MSI**. Но для начала разберёмся, что означает это слово из трёх букв.

Задание. Прочитайте в Википедии статью «Установщик Windows» и выясните, что такое **MSI**.

Нас интересует программа-установщик приложения для ОС **Microsoft Windows.**

Мало того, нас интересует 32-разрядная версия библиотеки.

Даже если на компьютере 64-разрядная операционная система, мы всё-таки поставим 32-разрядную версию. И в среде разработки мы будем разрабатывать 32-разрядные приложения **Win32**.

Конечно, желающие могут установить что-нибудь 64-разрядное. Мы здесь просто разбираем сквозной работающий пример, на базе которого можно развить свои собственные идеи — в любом направлении.

Итак, находим что-нибудь 32-разрядное (рис. 2.3).

В названии файла видим знакомые ключевые слова:

— Win;

— 32;

— msi.



Рис. 2.3. Выбор 32-разрядного установщика

Задание. Найдите и скачайте 32-разрядный установщик формата MSI.

Скачиваем установщик и запускаем его (рис. 2.4).

Может потребоваться дополнительное согласие на установку, если **Windows 10** будет сомневаться в чистоте наших намерений и задавать разные вопросы.

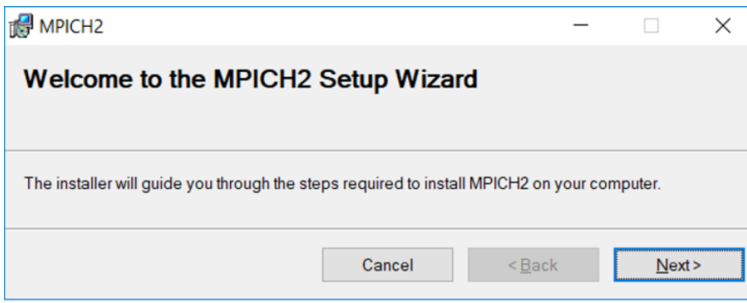


Рис. 2.4. Начало установки

Нажимаем **Next**. Появляется привычное окно установки программы (рис. 2.5).

Но не будем торопиться и щёлкать по кнопкам **OK — Next — Next — OK**.

В данном случае можно узнать много полезного, если пролистать текст и попытаться понять написанное (см. задание ниже).

Только после этого можно с облегчением вздохнуть и нажать-таки **Next**.

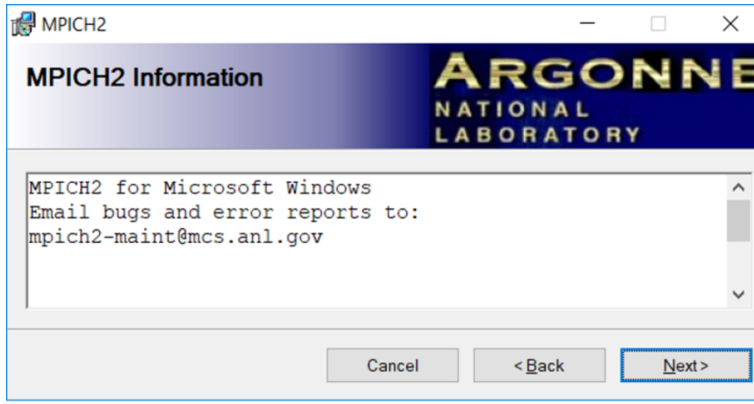


Рис. 2.5. Окно установщика

Задание. Просмотрите текст в окне установщика и выясните следующее:

— Какие компиляторы C/C++ поддерживает данная библиотека?

— Какие каталоги будут созданы?

— Где будут размещена библиотеки DLL?

— Какие настройки нужно сделать в компиляторе?

— Что нужно сделать при запуске программы на нескольких машинах?

— Как запустить параллельное приложение?

— В чём заключается минимальная установка?

— Как выполнить запуск из сетевого каталога?

В следующем окне появляется лицензионное соглашение (рис. 2.6).

Такой текст мало кто дочитывает до конца, а зря.

Здесь тоже можно кое-что для себя почерпнуть (см. задание).

Только после выполнения задания выбираем согласие с условиями **I agree** и нажимаем кнопку **Next**.



Рис. 2.6. Лицензионное соглашение

Задание. Изучите лицензионное соглашение и выясните следующее:

— Что нам разрешается делать с данной библиотекой?

— От каких претензий отказываются разработчики в разделе **Disclaimer**?

На следующем нам предлагают установить пароль **Pass Phrase (**рис. 2.7).

При этом пояснения занимают аж четыре строчки. И это не просто слова, а полезная информация.

Здесь нам сообщают несколько важных моментов:

Установку нужно делать с правами администратора.

Будет установлена системная служба (service) под названием **SMPD**. Она нужна для работы параллельного приложения MPI.

Пароль должен быть одинаковым на всех компьютерах, на которых будем запускать нашу параллельную программу.

Другими словами, мы запускаем одну программу, а выпол-

няются несколько процессов на нескольких компьютерах. Но работать они будут как одна большая программа. И мы это проделаем.

Пароль запомним на будущее. А лучше запишем на стикере и приклеим на монитор. Так обычно поступают начинающие пользователи. Так они надеются обеспечить безопасность компьютерной системы.

Чтобы легче было запомнить этот сложный пароль, можно послушать популярную песенку «Don't Worry, Be Happy» и почитать её историю где-нибудь в сети.

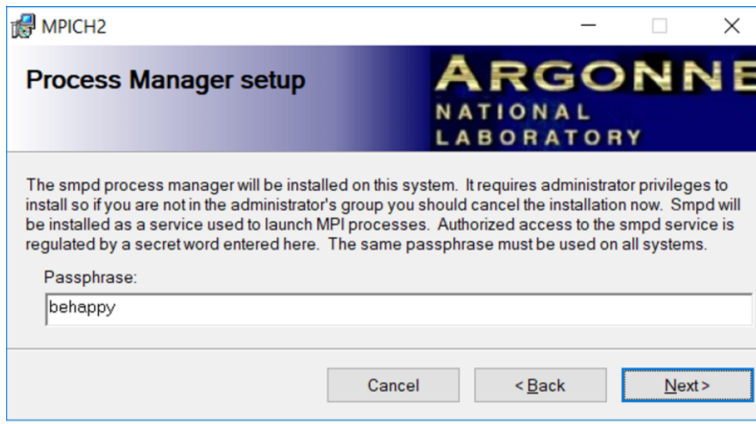


Рис. 2.7. Задание пароля

Задание. Установите и запишите пароль.

Судя по непроверенным сообщениям из достоверных источников в интернете, **SMPD** расшифровывается как **Simple Multi-Purpose Daemon**. Они же утверждают, что эта служба относится к средствам управления процессами — **Process managers.** Эта часть названия более-менее понятна.

Название заканчивается словом **Daemon**. Интересно, про каких таких «демонов» здесь идёт речь? Программисты обожают

образные названия. А ещё в компьютерах встречаются «сироты», «зомби» и другие явления природы.

Задание. Прочитайте в Википедии статью «Демон (программа)» и выясните, что компьютерщики называют словом «демон» и как появился этот термин.

Нажимаем кнопку **Next**.

Получаем новое диалоговое окно (рис. 2.8).

Мы выбираем каталог для установки библиотеки.

Обратим внимание на выбор по умолчанию:

C:\Program Files (x86) \MPICH2

Это ещё одно подтверждение того, что мы устанавливаем именно 32-разрядную версию.

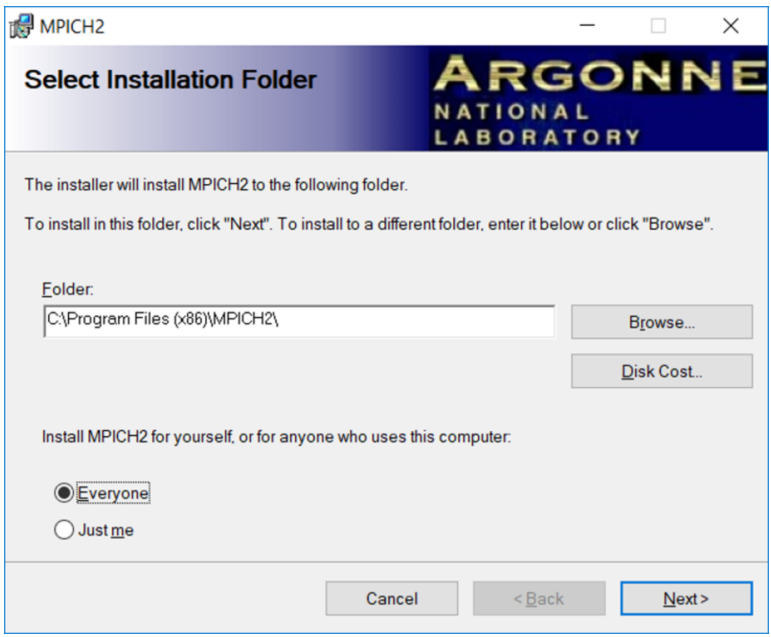


Рис. 2.8. Каталог для установки

Задание. Обратите внимание на каталог для установки и запишите его в отчёт.

Напомним, что нам желательно следить за разрядностью во всех её проявлениях:

— компьютер;
— операционная система;
— компилятор;
— библиотека;
— прикладная программа.

При этом на 64-разрядный компьютер можно установить 64-разрядную ОС. А прикладные программы могут быть и 32- и 64-разрядными.

Что бы всё окончательно запутать, прикладные программы устанавливаются в каталог **Program Files**.

Если у нас 64-разрядная ОС, в этот каталог будут установлены 64-разрядные приложения. А вот каталог **Program Files (x86)** предназначен для 32-разрядных программ (рис. 2.9).

При этом в 32-разрядной ОС программы 32-разрядные, и они устанавливаются в каталог **Program Files.**

В нашем примере (рис. 2.9) можно видеть, что компьютер и ОС 64-разрядные, а каталоги есть для обоих типов программ.



Рис. 2.9. Разрядность ОС и программ

Задание. Изучите свойства системы и каталоги для установки прикладных программ на своём компьютере.

Если говорить об установке приложений, **x86** прозрачно намекает на 32-разрядные программы. Эта традиция сложилась исторически. Дело было давно, когда невозможно было даже мечтать о 64 разрядах.

Задание. Прочитайте в Википедии статью «x86» и выясните, как появилось такое обозначение.

Нажимаем кнопки **Next — Next** и следим за процессом установки.

Читаем сообщение о завершении установки (рис. 2.10).

Похоже, в дальнейшем нам придётся установить ещё кое-что, но об этом потом.

Нажимаем кнопку **Close**.

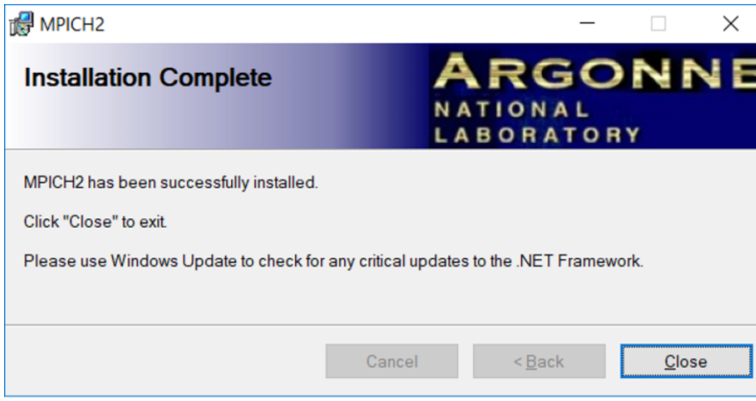


Рис. 2.10. Завершение установки

Задание. Установите библиотеку MPICH.

Рассмотрим содержимое каталога MPICH2 (рис. 2.11).

Нас будут интересовать три каталога:

— bin;

— include;

— lib.

Они нам понадобятся при настройке компилятора.

Ещё один полезный каталог называется **examples**. Судя по названию, здесь находятся примеры использования данной библиотеки. Примеры — во множественном числе. Угадайте с одного раза, сколько примеров можно найти в каталоге **examples**?

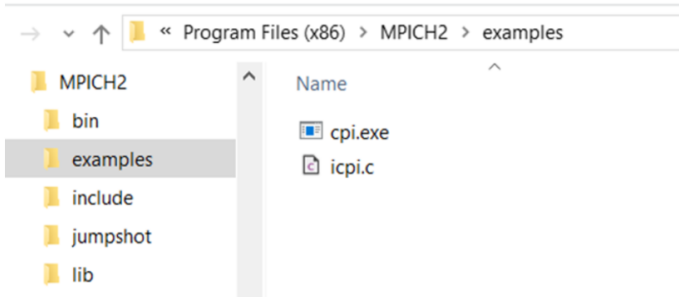


Рис. 2.11. Каталог MPICH2

Задание. Изучите содержимое каталога examples.

3. ПРОГРАММА «ВСЕМ ПРИВЕТ!»

Начнём знакомство с MPI с самой простой, стандартной программы «Hello, World!»

Для начала посмотрим видеоролик [16], чтобы представить себе общую схему дальнейшей работы. В этом разделе мы подробно разберём все необходимые шаги.

Задание. Посмотрите ролик [16].

Текст программы приводится на рис. 3.1.

Главная часть программы — вывод на экран короткого стандартного приветствия с помощью команды **printf** (строка 7).

Для работы функции **printf** подключаем заголовочный файл библиотеки стандартного ввода-вывода **STDIO — Standard Input/ Output** (строка 1).

Для организации параллельных процессов MPI мы добавляем к стандартной последовательной программе несколько строк.

Подключаем заголовочный файл библиотеки MPI (строка 2).

Вызываем инициализацию (начало работы) системы обмена сообщениями (строка 6):

MPI_Init.

В качестве аргументов функции инициализации используем два нулевых указателя **NULL**. Это своего рода «заглушка», фиктивные аргументы. В следующих примерах мы рассмотрим и сами аргументы.

В конце программы вызываем завершение работы системы обмена сообщениями (строка 6):

MPI_Finalize.

E:\Arkov-MPI\mpi1-hello\mpi1-hello.c

```
1  #include <stdio.h>
2  #include <mpi.h>
3
4  void main()
5  {
6      MPI_Init(NULL, NULL);
7      printf("Hello, World!\n");
8      MPI_Finalize();
9  }
```

Рис. 3.1. Первая программа

Программа готова, но не будем спешить и скорее нажимать комбинацию **Ctrl+F5.**

Для начала установим конфигурацию **Release** — в верхней части окна среды разработки **Visual Studio.** В данной работе мы не будем углубляться в технологию отладки программ. Сами программы будут простыми. Нас будет интересовать технология распараллеливания и эффективность работы.

Задание. Составьте программу (рис. 3.1) и установите конфигурацию **Release**.

3.1. НАСТРОЙКА КАТАЛОГА INCLUDE

Чтобы скомпилировать параллельную программу, нам понадобится сделать три настройки. Эти же настройки придётся сделать и для всех последующих проектов с применением MPI.

Переходим к настройке свойств проекта, а именно, каталогов для нашей библиотеки MPI:

Project — Properties — Configuration Properties — VC++ Directories.

Первая настройка — расположение каталога **Include**.

Выбираем раздел **General** и находим пункт **Include Directories (**рис. 3.2).

Справа от списка каталогов находим кнопку с символом стрелки вниз и щёлкаем по ней. Появляется выпадающее меню с одним пунктом:

<Edit...>.

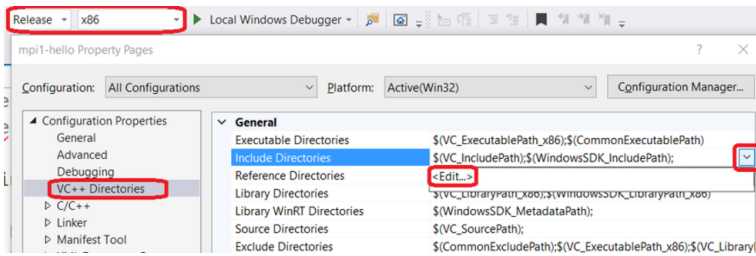


Рис. 3.2. Настройка каталога**Include**

Появляется диалоговое окно **Include Directories** (рис. 3.3).

Наводим курсор на кнопку с символом каталога.

Всплывающая подсказка сообщает, что эта кнопка позволяет вставить новую строчку в список каталогов **Include**.

Щёлкаем по этой кнопке.

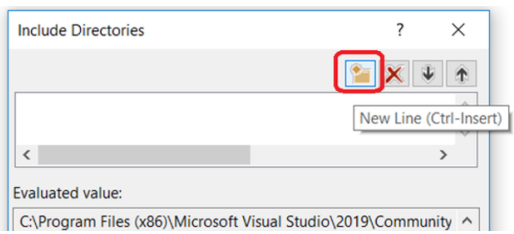


Рис. 3.3. Вставка новой строки

Появляется пустая строка, где можно что-нибудь напеча-

тать. Но это если мы точно помним полный путь к нужной папке.

Поступим проще. Находим в правом конце пустой строки кнопку с многоточием (рис. 3.4). Щёлкаем по этой кнопке.

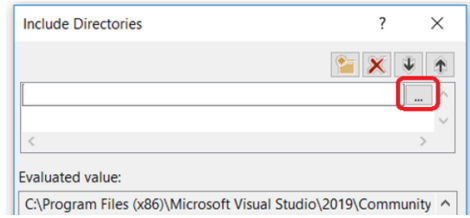


Рис. 3.3. Вставка новой строки

Появляется диалоговое окно
Select Directory.
Переходим на диск **C**:. Находим каталог библиотеки MPI:
C:\Program Files (x86) \MPICH2
Выбираем каталог **include**.
Читаем надпись на кнопке:
Select Folder.

Обратим внимание, что в заголовке окна говорится, что мы выбираем **Directory**. Но после выбора каталога на кнопке написано про выбор **Folder** (рис. 3.5).

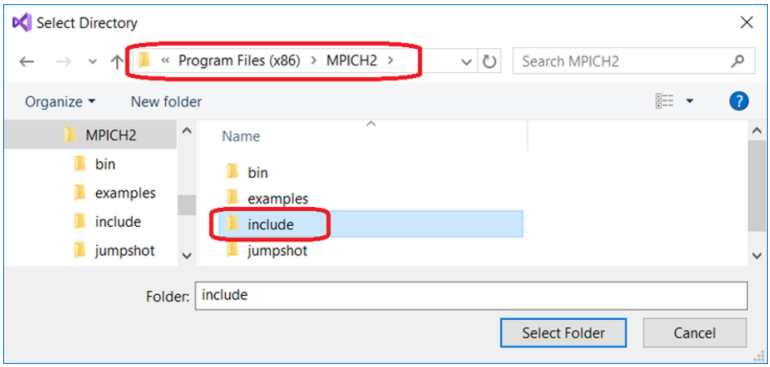


Рис. 3.5. Выбор каталога**Include**

Задание. Прочитайте в Википедии начало статьи на русском языке «Каталог (файловая система)» и статьи на английском языке «Directory (computing)» и выясните, какими словами может называться этот объект файловой системы.

Нажимаем кнопку **Select Folder.**

Мы вернулись в предыдущее окно **Include Directories** (рис. 3.6).

Наш каталог появился в строке для ручного ввода.

Тот же каталог появился и строке чуть ниже, в разделе **Evaluated Value.**

Обратим внимание, что в первом случае вместо круглых скобок появились загадочные выражения **%28** и **%29**.

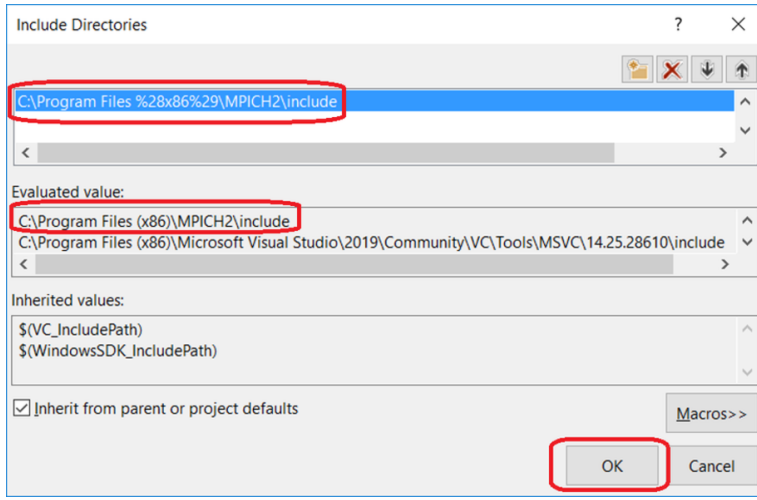


Рис. 3.6. Каталог**Include**

Задание. Просмотрите в Википедии статью «Скобки» и выясните, в какой кодировке для круглых скобок используют коды **28** и **29**.

Нажимаем **OK**.

Мы вернулись в окно настройки свойств проекта (рис. 3.7).

В строке **Include Directories** появился наш каталог.

В нижней части окна выводится пояснение о том, зачем нужна эта настройка.

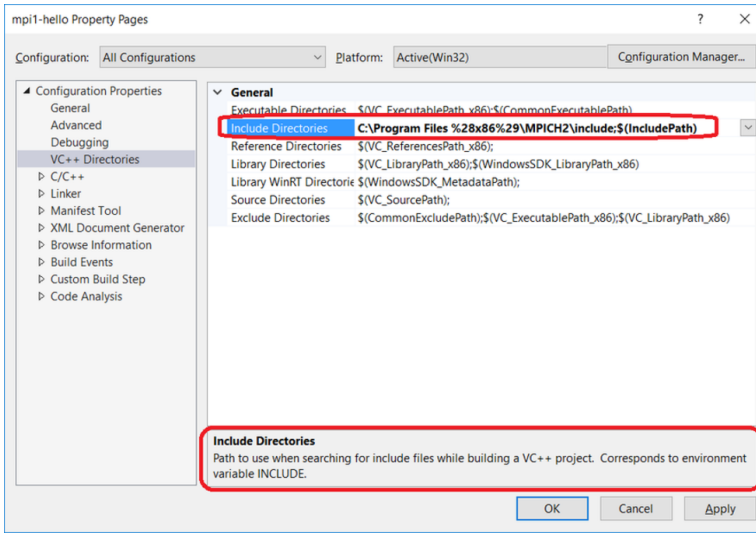


Рис. 3.7. Каталог**Include**настроен

Задание. Изучите пояснение по поводу **Include Directories** в нижней части окна **Property Pages.**

Задание. Настройте каталог **Include** в свойствах проекта.

3.2. НАСТРОЙКА КАТАЛОГА LIB

Мы справились с первой настройкой проекта.

Теперь на очереди вторая настройка — каталог **Lib**.

Щёлкаем по строке **Library Directories.**

Пояснения внизу окна изменились. Почитаем, ознакомимся (рис. 3.8).

Затем нажимаем кнопку со стрелочкой в правом конце строки и щёлкаем по кнопке **<Edit...>**.

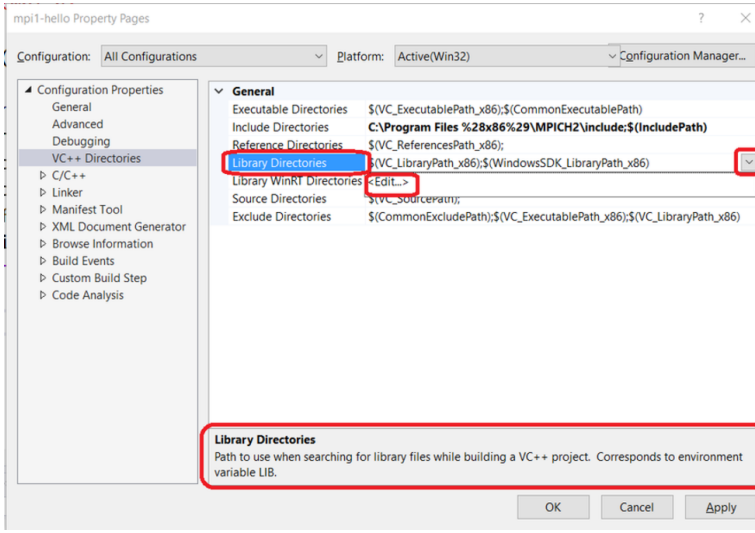


Рис. 3.8. Настройка каталога **Lib**

Задание. Изучите пояснение по поводу **Library Directories** в нижней части окна **Property Pages.**

Далее вся история повторяется.
Мы добираемся до окна выбора каталога (рис. 3.9).
Указываем, где находится каталог **lib**.

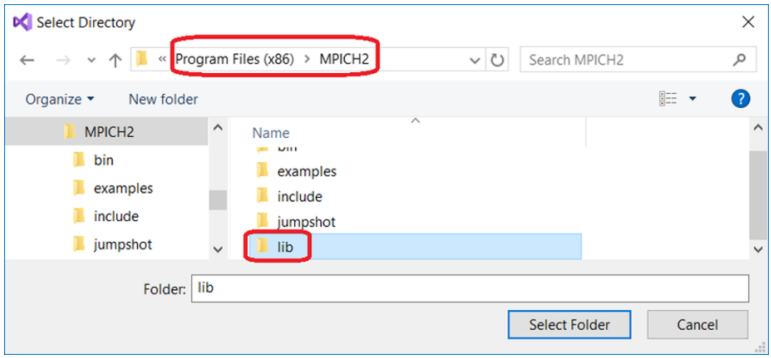


Рис. 3.9. Выбор каталога **lib**

Убеждаемся, что каталог **lib** выбран и что он теперь выводится в списке **Library Directories** (рис. 3.10).

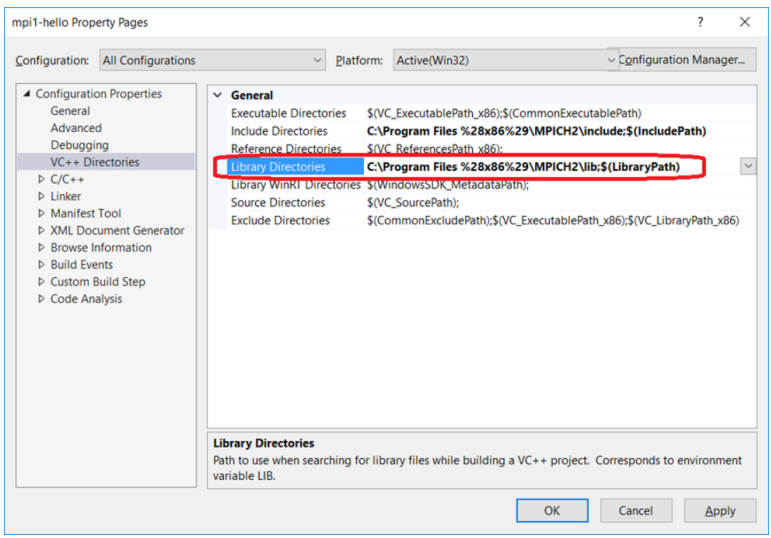


Рис. 3.11. Каталог**lib**выбран

Задание. Настройте каталог **lib** в свойствах проекта.

3.3. ПОДКЛЮЧЕНИЕ БИБЛИОТЕКИ MPI.LIB

Наконец, третья настройка — подключение библиотеки **mpi.lib**.

В диалоговом окне настройки свойств проекта **Property Pages** выбираем следующий пункт:

Configuration Properties — Linker — Input — Additional Dependencies.

В нижней части окна читаем краткие пояснения. Здесь упоминается загадочный **link**.

Щёлкаем по знакомой кнопке со стрелочкой (рис. 3.12).

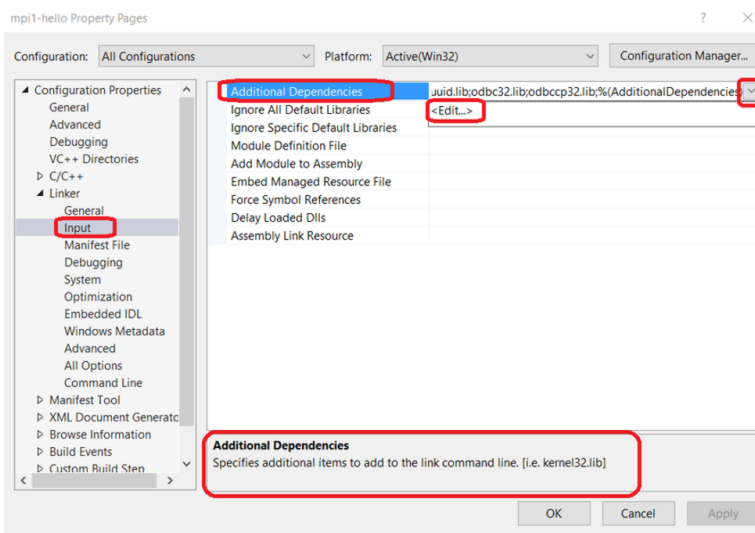


Рис. 3.12. Дополнительная библиотека

Задание. Просмотрите в Википедии статью на английском языке «Linker (computing)» и статью на русском языке «Компо-

новщик» и выясните, что такое LINKER.

Щёлкаем по строчке **<Edit...>.**
Появляется диалоговое окно **Additional Dependencies.**
Вручную вводим название библиотеки:
mpi.lib.
Нажимаем **OK** (рис. 3.13).

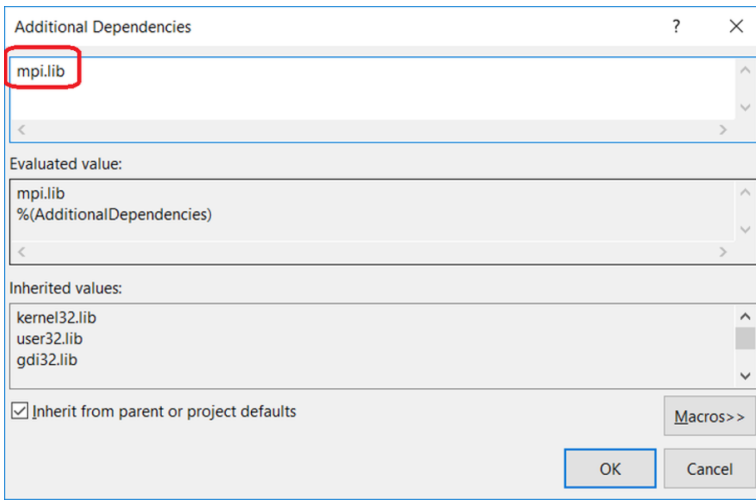


Рис. 3.13. Ввод названия библиотеки вручную

Возвращаемся в окно свойств проекта.
Убеждаемся, что наша третья настройка принята (рис. 3.14).

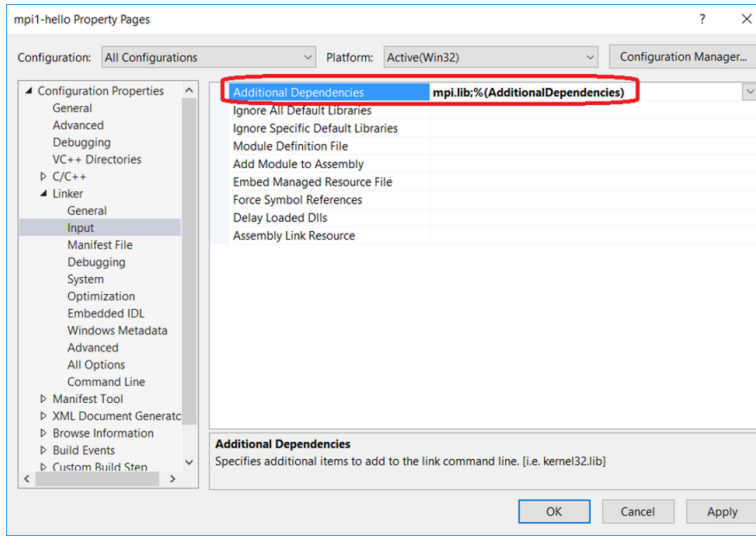


Рис. 3.14. Название библиотеки введено

Задание. Настройте подключение библиотеки **mpi.lib** в свойствах проекта.

3.4. ЗАПУСК ПРОГРАММЫ

Все настройки сделаны.

Наконец-то можно запускать программу.

Нажимаем комбинацию клавиш:

Ctrl+F5.

В командном окне отладчика видим знакомое приветствие (рис. 3.15).

Перед нами обычная последовательная программа.

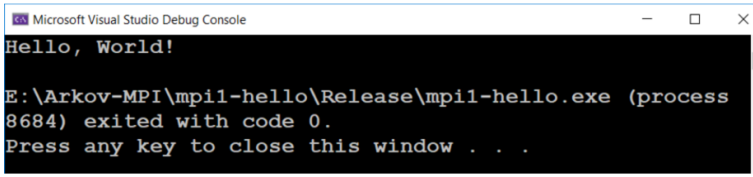


Рис. 3.15. Запуск из среды разработки

Задание. Запустите программу из среды разработки.

Обратим внимание на название в заголовке командного окна:

Debug Console.

Слово «консоль» часто встречается у компьютерщиков.

Пришло время познакомиться с этим термином поподробнее.

Задание. Изучите перечисленные источники и выясните значение слова «консоль» (как строительная конструкция и как компьютерное устройство), а также его происхождение (первоначальный смысл слова):

— Викисловарь — консоль: https://ru.wiktionary.org/wiki/
— Wiktionary — console: https://en.wiktionary.org/wiki/
— Википедия — «Консоль (архитектура)», «Консоль»
— Wikipedia — «Console (architecture)», «System console»

Запустим программу из командного окна.

Открываем новое окно:

Win+R — cmd.

Переходим на рабочий диск.

Переходим в каталог с исполняемым файлом в конфигурации **Release**.

Запускаем файл на выполнение. Расширение **EXE** указывать не обязательно.

Получаем привычный результат — сообщение от последовательной программы (рис. 3.16).

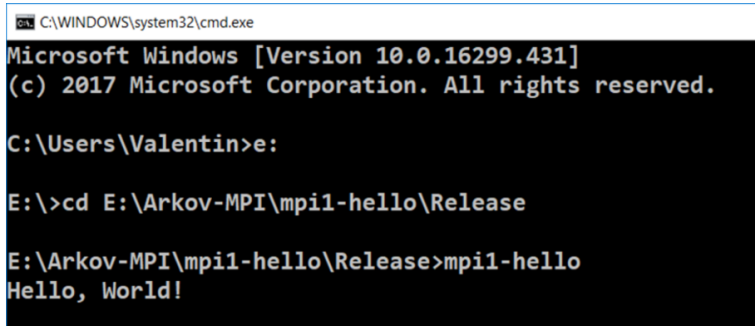


Рис. 3.16. Запуск последовательной программы

Задание. Запустите последовательную программу в командном окне.

Чтобы запустить параллельное приложение, нам понадобится специальная программа–утилита:

mpiexec.

Название программы говорит о том, что она позволяет взять параллельное приложение MPI и запустить его на выполнение — EXECUTE.

Поскольку мы работаем в командной строке, нам нужно сообщить операционной системе, где находится эта программа.

При установке библиотеки MPI мы запомнили, где она прячется (рис. 3.17). Мы знаем, где искать нашу утилиту. Это каталог **bin**, то есть **binary files.** Другими словами, исполняемые фалы пакета MPICH2.

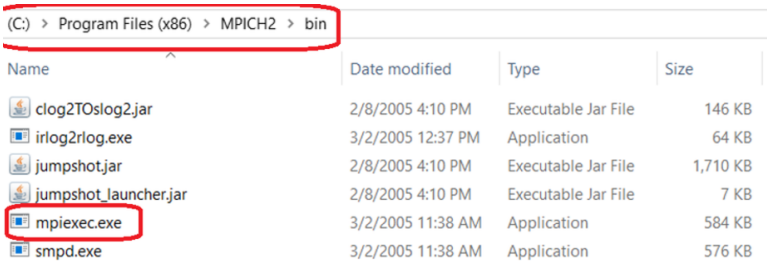


Рис. 3.17. Расположение утилиты

Чтобы запустить файл на выполнение, нужно указать полный путь к файлу. Но так работать слишком неудобно.

Проще добавить новый путь в переменную окружения **Path**.

Щёлкнем по строке адреса в **Проводнике** и скопируем путь к нашему файлу в буфер обмена (рис. 3.18).

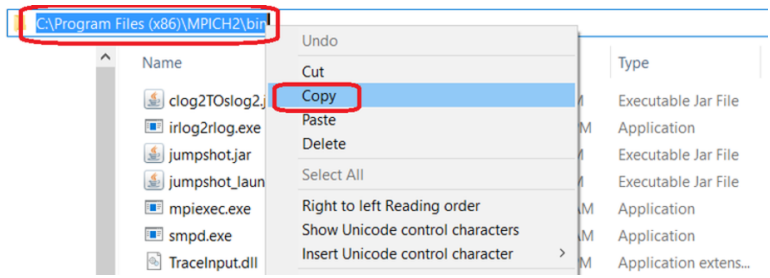


Рис. 3.18. Копирование пути

Возвращаемся в командное окно.

Посмотрим значение переменной окружения **Path**:

set Path.

Обратим внимание, что пути в списке разделены точкой с запятой (рис. 3.19).

Попутно мы можем увидеть список расширений имён исполняемых файлов:

PATHEXT

Сокращение **EXT** скрывает фразу **File name extension.**

Добавляем путь для поиска в переменную **Path**:

set path=%path%C:\Program Files (x86) \MPICH2\bin

Мы используем имеющееся значение переменной **%path%.**

После него дописываем новый путь для поиска исполняемых файлов. Точнее, вставляем из буфера обмена.

Выводим на экран новое значение переменной:

set path.

Убеждаемся, что предыдущие настройки не пострадали и что новый путь добавлен без ошибок.

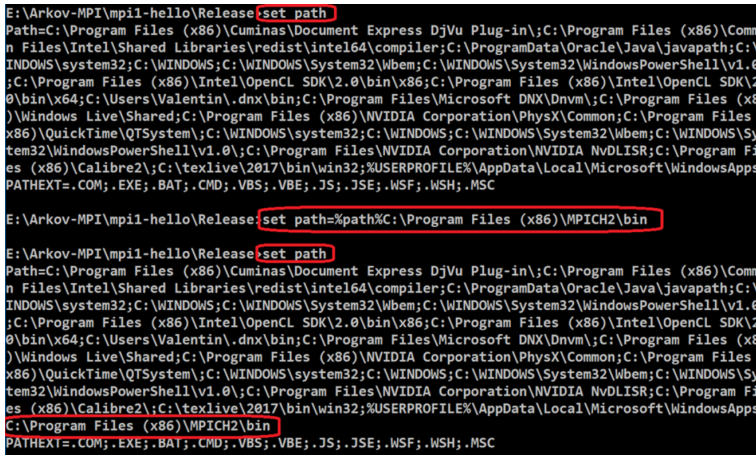


Рис. 3.19. Добавление пути поиска

Задание. Добавьте путь для поиска в переменную **path**.

Теперь мы можем запустить программу **mpiexec**.

На экран выводится подробное описание параметров запуска утилиты (рис. 3.20). Другими словами, перед нами формат запуска команды.

В квадратных скобках указаны необязательные аргументы, без которых можно обойтись.

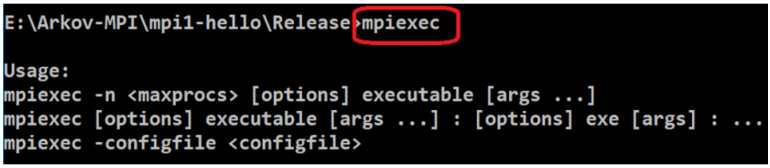


Рис. 3.20. Формат команды**mpiexec**

Задание. Изучите описание команды **mpiexec**.

В первой строке описания показано, как запускать параллельное приложение.

В простейшем случае формат запуска такой:

mpiexec -n <maxprocs> executable.

После ключа **-n** мы указываем количество параллельных процессов.

Затем сообщаем имя исполняемого файла.

Запустим два процесса и укажем наш исполняемый файл:

mpiexec -n 2 mpi1-hello.

При запуске mpiexec может потребоваться ввести пароль, который мы видели при установке MPI.

Напомним, что по умолчанию используется следующий пароль:

behappy.

Не забываем переключиться на английский язык при вводе пароля. После ввода пароля запускается указанное количество потоков (рис. 3.21).

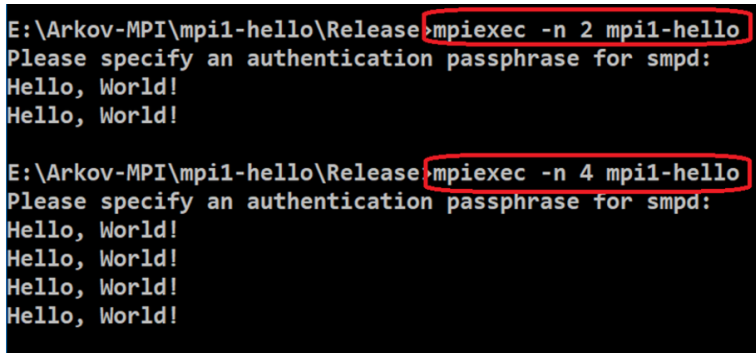


Рис. 3.21. Запуск параллельного приложения

Задание. Запустите параллельное приложение с 2, 4 и 6 процессами.

4. НОМЕР ПРОЦЕССА

В предыдущей работе мы уже определяли номер потока в группе. Здесь мы определим номер процесса в группе. Это нужно для распределения заданий между параллельными процессами.

Чтобы разобраться с очередным примером (рис. 4.1), придётся определиться с красивыми названиями.

При запуске параллельного приложения создаётся группа параллельных процессов, внутри которой организован обмен сообщениями. Таких групп может быть несколько. Для выбора конкретной группы используется специальная константа под названием КОММУНИКАТОР. Можно сказать, что коммуникатор определяет конкретную группу процессов и некоторые свойства этой группы. Каждый процесс внутри группы (с конкретным коммуникатором) получает свой индивидуальный номер, начиная с нуля.

Если выразиться очень упрощённо, можно считать, что коммуникатор — это название группы параллельных процессов.

По умолчанию, все параллельные процессы, запущенные с помощью mpiexec, входят в группу с коммуникатором MPI_COMM_WORLD.

В этом названии на первой позиции указана технология MPI. Далее идёт слегка сокращённое слово COMMUNICATOR. И наконец, завершает название слово WORLD. Как и в названии первой программы «Hello, World», это слово здесь означает «все». То есть «все процессы». Все процессы, созданные при запуске параллельного приложения (прикладной программы).

Каждый процесс может узнать свой номер в рамках группы с указанным коммуникатором:

MPI_Comm_rank (MPI_COMM_WORLD, &ProcessRank).

Здесь слово RANK (по-русски РАНГ) означает порядковый номер процесса в группе (коммуникаторе).

Кроме того, каждый процесс может узнать, сколько всего процессов в указанном коммуникаторе:

MPI_Comm_size (MPI_COMM_WORLD, &CommunicatorSize).

Размер коммуникатора — это количество процессов в указанной группе.

Обе функции возвращают указатели на выходные переменные.

Каждый процесс выводит на экран свой порядковый номер и общее количество параллельных процессов (строка 10).

E:\Arkov-MPI\mpi2-rank\mpi2-rank.c

```
 1  #include <stdio.h>
 2  #include <mpi.h>
 3
 4  void main()
 5  {
 6      int ProcessRank, CommunicatorSize;
 7      MPI_Init(NULL, NULL);
 8      MPI_Comm_rank(MPI_COMM_WORLD, &ProcessRank);
 9      MPI_Comm_size(MPI_COMM_WORLD, &CommunicatorSize);
10      printf("%d of %d\n", ProcessRank, CommunicatorSize);
11      MPI_Finalize();
12  }
```

Рис. 4.1. Определяем номер процесса

Задание. Составьте программу (рис. 4.1).

Снова делаем три настройки:

— include;
— lib;
— mpi.lib.

Эти три настройки придётся делать для каждого проекта.

У нас будет не так много проектов, чтобы об этом переживать.

Компилируем программу и убеждаемся в работоспособности последовательной программы:

Ctrl+F5.

Затем запускаем программу в командном окне (рис. 4.2).

Если спросят, сообщаем секретный пароль.

Обратим внимание на порядок появления сообщений на экране.

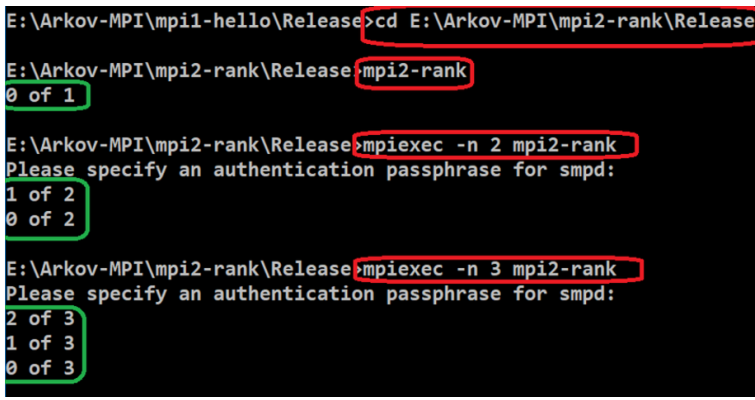


Рис. 4.2. Последовательный и параллельный запуск

Задание. Запустите программу в последовательном и параллельном режимах несколько раз и обратите внимание на порядок вывода сообщений.

5. ПАРАМЕТРЫ КОМАНДНОЙ СТРОКИ

Как и в предыдущей работе, мы будем управлять работой параллельной программы через параметры командной строки. Это нужно для многократного запуска приложения из пакетного файла. В конечном счёте, мы определим ускорение и эффективность распараллеливания.

Для начала нужно освежить в памяти основные моменты ввода параметра командной строки и преобразования строки в число. На эту тему у нас появился видеоролик [17].

Задание. Посмотрите ролик [17].

Мы вводим в программу один параметр командной строки (рис. 5.1).

Для упрощения примера мы не проверяем количество параметров. Если при запуске программы не будет ни одного параметра, то произойдёт ошибка. Если параметров будет много, программа их проигнорирует

Поэтому при объявлении основного модуля программы указываем две входные переменные (строка 5):

void main (int argc, char* argv []), где

— **argc** — количество параметров;

— **argv** — массив строк — значений параметров.

Мы не возвращаем никаких значений, поэтому объявляем тип void. За счёт этого мы сократим исходный текст на одну строчку. Конечно, потом компилятор по умолчанию добавит возврат значения ноль. Так что ничего не изменится.

При инициализации системы обмена сообщениями мы передаём всем процессам указатели на количество и значения параметров командной строки (строка 8).

Преобразуем первый параметр из строки символов в длинное целое число (строка 10):

N = atoll (argv [1]).

В нашем примере длинное целое имеет тип **long long** и занимает 8 байт в памяти. Соответственно, можно легко оценить диапазон возможных значений такой переменной. Мы это у же делали в предыдущей работе.

Для использования данной функции подключаем соответствующую библиотеку (строка 3):

#include <stdlib. h>.

Выводим на экран номер процесса (строка 12).

Печатаем введённое число, используя формат вывода **%lld** для длинного целого (строка 13). Всё это должно быть уже знакомо по предыдущим работам.

E:\Arkov-MPI\mpi3-arg\mpi3-arg.c

```
 1  #include <stdio.h>
 2  #include <mpi.h>
 3  #include <stdlib.h>
 4
 5  void main (int argc, char* argv[])
 6  {
 7      int ProcessRank;
 8      MPI_Init (&argc, &argv);
 9      long long N;
10      N = atoll(argv[1]);
11      MPI_Comm_rank (MPI_COMM_WORLD, &ProcessRank);
12      printf ("%d:\t", ProcessRank);
13      printf("%lld\n", N);
14      MPI_Finalize ();
15  }
```

Рис. 5.1. Ввод параметра командной строки

Задание. Составьте программу (рис. 5.1).

Делаем три стандартные настройки для подключения биб-

лиотеки MPI.

Компилируем проект.

Переходим в командное окно и запускаем нашу программу.

После названия исполняемого файла через пробел указываем одно число через пробел.

Числа распознаются.

При выходе за пределы возможных значений получаем максимальное число (рис. 5.2).

```
E:\Arkov-MPI\mpi3-arg\Release>mpi3-arg 1000000000000000
0:      1000000000000000

E:\Arkov-MPI\mpi3-arg\Release>mpi3-arg 10000000000000000
0:      10000000000000000

E:\Arkov-MPI\mpi3-arg\Release>mpi3-arg 100000000000000000000
0:      9223372036854775807
```

Рис. 5.2. Запуск последовательной программы

Задание. Запустите последовательную программу и определите пределы значений параметра.

Последовательная программа работает как обычно.

Чтобы запустить параллельную программу, нужна утилита **mpiexec**.

Поскольку работу мы продолжаем на следующий день, а то и через месяц, перед сессией, все настройки в командной строке давно обнулились.

Чтобы не вводить путь для поиска утилиты каждый раз, сделаем одну общую настройку для всей операционной системы.

Запустим панель управления:

Settings.

Задаём поиск **path** и получаем окно свойств системы:

System Properties (рис. 5.3).

Находим вкладку дополнительных настроек:

Advanced.

Между прочим, это окно нисколько не изменилось со времён **Windows XP**, а то и более ранних версий операционной системы. Меняется только самый верхний, внешний уровень интерфейса. И действительно, если всё работает — зачем что-то менять?

В нижней части окна нажимаем кнопку настройки переменных окружения:

Environment Variables.

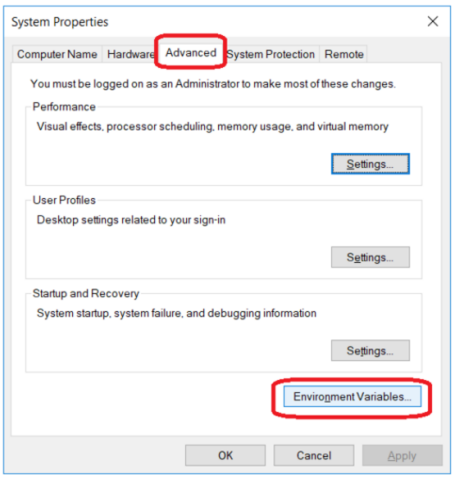


Рис. 5.3. Свойства системы

Появляется диалоговое окно **Environment Variables** (рис. 5.4).

Находим раздел **System variables** в нижней половине окна.

Выделяем строчку переменной, отвечающей за путь поиска исполняемых файлов:

Path.

Щёлкаем кнопку редактирования параметра:

Edit.

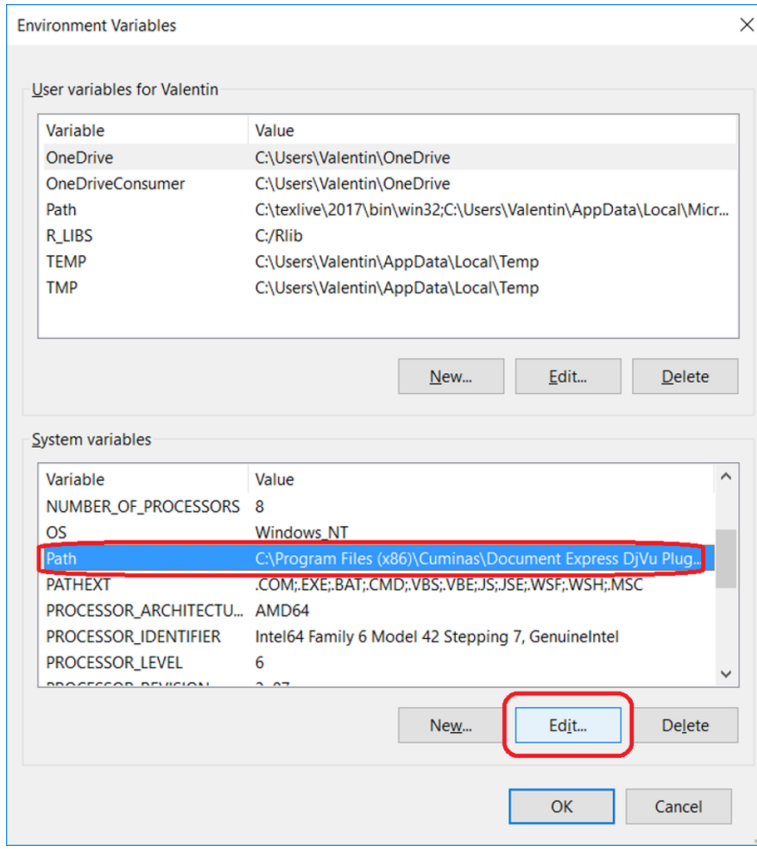


Рис. 5.4. Настройка параметров окружения

Появляется диалоговое окно редактирования переменой окружения:

Edit environment variable (рис. 5.5).

Переходим в **Проводник Explorer** и копируем в буфер обмена путь к каталогу исполняемых файлов **bin** библиотеки MPICH2. Мы это уже проделывали в предыдущем разделе.

Нажимаем кнопку вставки нового пути для поиска:

New.

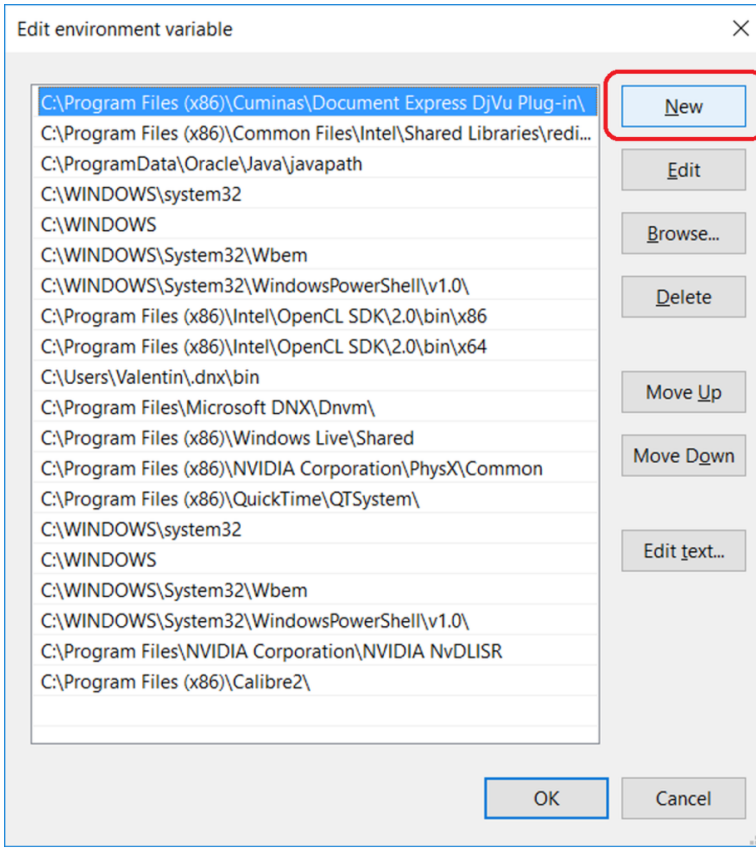


Рис. 5.5. Вставка нового пути

Вставляем новый путь из буфера обмена.
Нажимаем клавишу ввода:
Enter.
Наш путь добавлен (рис. 5.6).
Нажимаем кнопки **OK — OK — OK** и закрываем все окна.

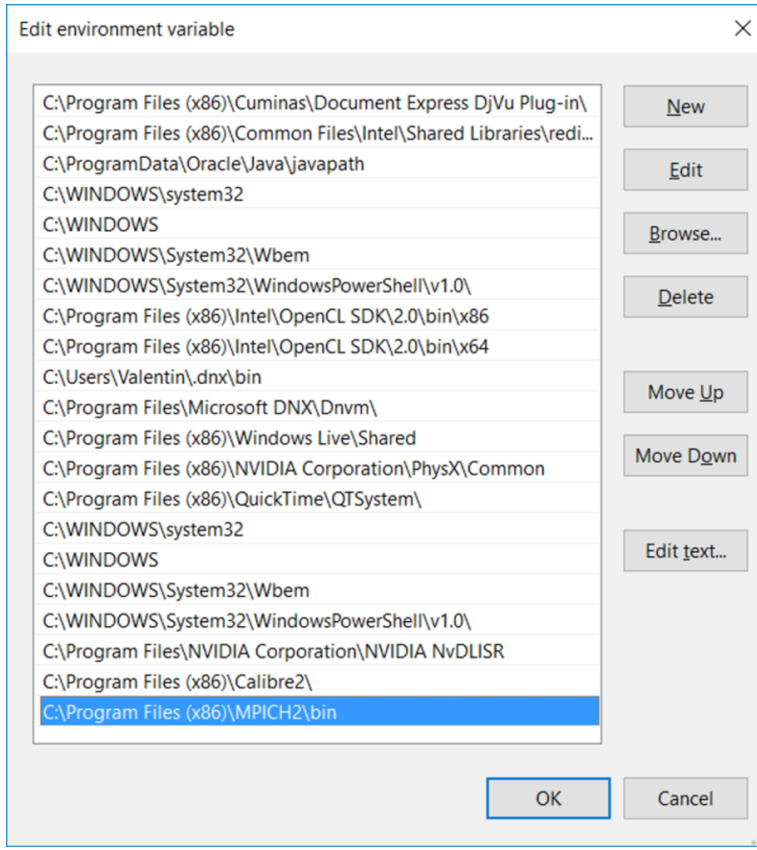


Рис. 5.6. Новый путь добавлен

Задание. Добавьте путь для поиска.

Закрываем командное окно.

На него новые настройки никак не повлияют.

Новое значение параметра **Path** сработает только для программ, запущенных после внесения изменений.

Открываем новое командное окно.

Выводим на экран значение переменной **Path** (рис. 5.7).

Убеждаемся, что путь для поиска добавлен в общий список.

```
C:\Users\Valentin>path
PATH=C:\Program Files (x86)\Cuminas\Document Express DjVu Plug-in\;C:\Program File
s (x86)\Common Files\Intel\Shared Libraries\redist\intel64\compiler;C:\ProgramData
\Oracle\Java\javapath;C:\WINDOWS\system32;C:\WINDOWS;C:\WINDOWS\System32\Wbem;C:\W
INDOWS\System32\WindowsPowerShell\v1.0\;C:\Program Files (x86)\Intel\OpenCL SDK\2.
0\bin\x86;C:\Program Files (x86)\Intel\OpenCL SDK\2.0\bin\x64;C:\Users\Valentin\.d
nx\bin;C:\Program Files\Microsoft DNX\Dnvm\;C:\Program Files (x86)\Windows Live\Sh
ared;C:\Program Files (x86)\NVIDIA Corporation\PhysX\Common;C:\Program Files (x86)
\QuickTime\QTSystem\;C:\WINDOWS\system32;C:\WINDOWS;C:\WINDOWS\System32\Wbem;C:\WI
NDOWS\System32\WindowsPowerShell\v1.0\;C:\Program Files\NVIDIA Corporation\NVIDIA
NvDLISR;C:\Program Files (x86)\Calibre2\;C:\Program Files (x86)\MPICH2\bin;C:\texl
ive\2017\bin\win32;%USERPROFILE%\AppData\Local\Microsoft\WindowsApps;
```

Рис. 5.7. Путь добавлен

Задание. Убедитесь, что путь добавлен.

Переходим на рабочий диск (рис. 5.8).

Заходим в каталог, где нас ожидает скомпилированная программа.

Запускаем параллельное приложение:

mpiexec -n 2 mpi3-arg 100.

Здесь мы вызываем утилиту **mpiexec**, которая нам запустит наше параллельное приложение.

С помощью ключа **-n** сообщаем количество параллельных процессор, которые нужно будет запустить.

Затем следует имя исполняемого файла.

После него через пробел вводим параметр — целое число.

Этот приём мы будем использовать, чтобы задать количество итераций в цикле.

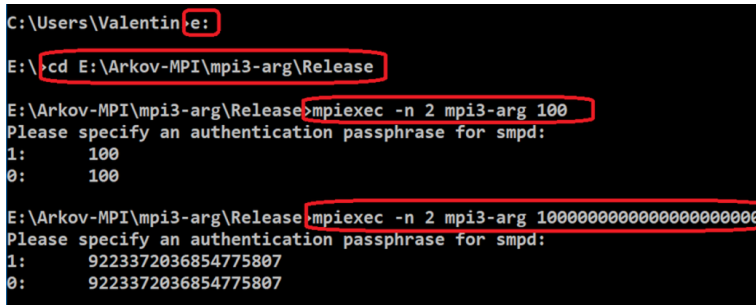


Рис. 5.8. Запуск параллельного приложения

Задание. Запустите параллельное приложение с разным количество процессов и убедитесь, что параметр командной строки вводится во все процессы.

6. ПАРАЛЛЕЛЬНОЕ СУММИРОВАНИЕ

Рассмотрим пример сложения большого количества единичек (рис. 6.1). Такой пример мы тоже рассматривали в предыдущих работах.

В командной строке мы указываем количество итераций цикла (строка 5).

Этот параметр передаётся во все параллельные процессы (строка 8).

Каждый параллельный процесс в цикле находит сумму указанного количества единичек (строки 12 и 13).

Затем он выводит заданное количество итераций и полученную сумму на экран (строка 15).

E:\Arkov-MPI\mpi4-sum\mpi4-sum.c

```
 1  #include <stdio.h>
 2  #include <mpi.h>
 3  #include <stdlib.h>
 4
 5  void main(int argc, char* argv[])
 6  {
 7      int ProcessRank;
 8      MPI_Init(&argc, &argv);
 9      long long N, i, S = 0;
10      N = atoll(argv[1]);
11      MPI_Comm_rank(MPI_COMM_WORLD, &ProcessRank);
12      for (i = 0; i < N; i++)
13          S++;
14      printf("%d:\t", ProcessRank);
15      printf("%lld\t%lld\n", N, S);
16      MPI_Finalize();
17  }
```

Рис. 6.1. Параллельная сумма

Задание. Составьте программу (рис. 6.1).

Программа готова.

Нам снова предстоит сделать три стандартные настройки.

Немного облегчим себе работу.

Создадим текстовый файл и сохраним в нём пути к каталогам **include** и **lib** (рис. 6.2).

Затем можно будет просто копировать эти строки в буфер обмена и вставлять при настройке свойств проекта.

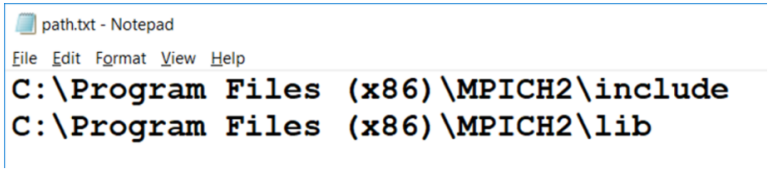


Рис. 6.2. Пути для настройки проекта

Задание. Сформируйте вспомогательный текстовый файл для настройки свойств проекта.

Вначале запускаем исполняемый файл как ПОСЛЕДОВАТЕЛЬНОЕ приложение с указанием числа итераций (рис. 6.3). Затем запускаем ПАРАЛЛЕЛЬНОЕ приложение с указанием количества процессов и итераций.

Каждый раз результаты выводятся без ошибок.

В случае параллельных процессов гораздо реже встречается ситуация гонки.

У каждого процесса свои переменные, и эти области памяти не пересекаются. Один процесс не сможет просто так взять и затереть чужие результаты.

Конечно, можно организовать ситуацию гонки с помощью специальных функций MPI для обмена данными, но мы этим пока не будем заниматься.

Итак, каждый процесс посчитал свою сумму и вывел её на экран. Это полностью независимая работа процессов.

```
E:\Arkov-MPI\mpi4-sum\Release>mpi4-sum 100
0:      100     100

E:\Arkov-MPI\mpi4-sum\Release>mpiexec -n 4 mpi4-sum 100
Please specify an authentication passphrase for smpd:
0:      100     100
3:      100     100
2:      100     100
1:      100     100

E:\Arkov-MPI\mpi4-sum\Release>mpiexec -n 3 mpi4-sum 1000000000
Please specify an authentication passphrase for smpd:
0:      1000000000      1000000000
1:      1000000000      1000000000
2:      1000000000      1000000000
```

Рис. 6.3. Параллельный запуск

Задание. Запустите параллельное приложение с разным количеством процессов.

Теперь добьёмся полной загрузки всех ядер.

Запустим число процессов по числу виртуальных процессоров.

Подберём число итераций так, чтобы компьютер проработал несколько десятков секунд.

Запустим **Диспетчер задач:**

Task Manager.

Открываем вкладку **Быстродействие:**

Performance (рис. 6.4).

Общая загрузка процессора составляет 100%.

Загрузка каждого из восьми логических процессоров тоже 100%.

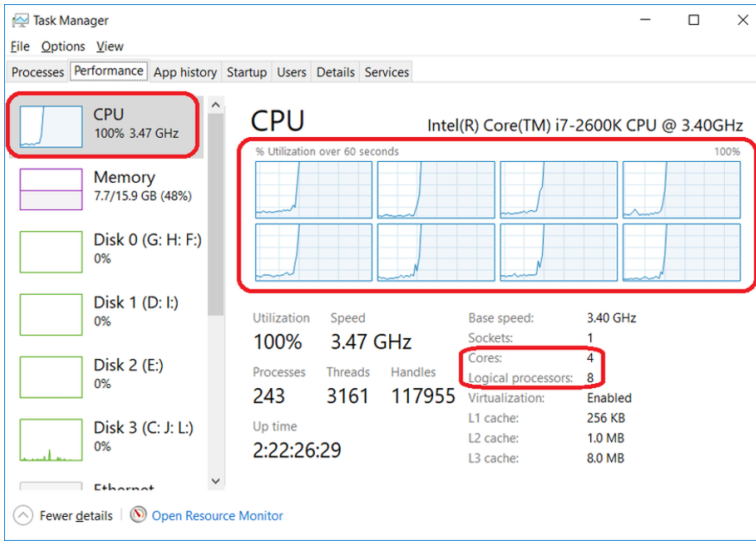


Рис. 6.5. Загрузка процессора

Задание. Запустите параллельное приложение так, чтобы обеспечить полную загрузку процессора.

Переходим на вкладку **Процессы:**

Processes.

Рассмотрим список процессов, которые выполняются прямо сейчас (рис. 6.6).

Мы обнаруживаем восемь процессов, как мы и просили. Все они указывают на один и тот же исполняемый файл.

Кроме того, мы находим менеджер процессов MPI:

MPICH2 Process Manager.

Этот процесс обслуживает систему обмена сообщениями. Здесь же указано, что это не просто процесс, а системная служба (Service).

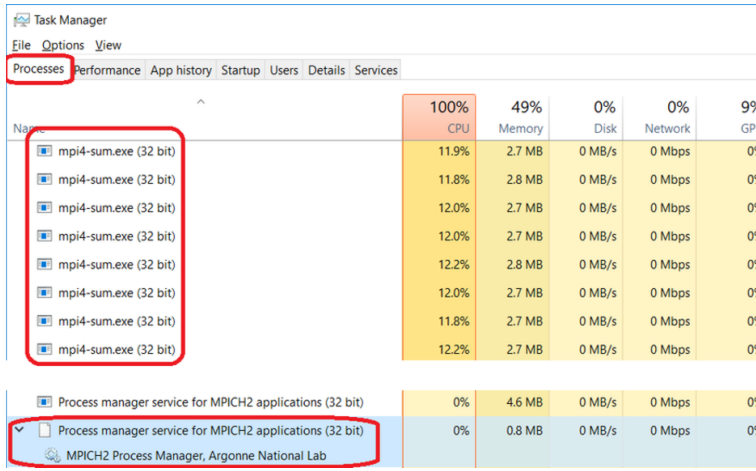


Рис. 6.6. Процессы

Задание. Во время работы параллельного приложения изучите список запущенных процессов.

7. РЕДУКЦИЯ

Мы рассмотрели независимую работу параллельных процессов. Теперь попробуем их объединить для решения одной общей задачи (рис. 7.1).

Наша программа суммирования ещё немного усложнилась.

Для раздачи заданий мы используем метод чередования. Он напоминает раздачу игральных карт по кругу — пока не закончится колода.

В каждом процессе счётчик цикла **i** начинается с номера процесса и изменяется с шагом, равным количеству процессов (строка 13).

В нашем примере будет восемь процессов с номерами от 0 до 7. Счётчик цикла процесса номер 0 получает следующие значения: 0, 8, 16, 32 и так далее. Счётчик цикла процесса номер 1 получает следующие значения: 1, 9, 17, 33 и так далее. В результат каждый процесс получит примерно равное количество заданий.

Каждый процесс в цикле находит свою часть общей суммы (строка 14).

Затем каждый процесс выводит на экран свой номер и свою частичную сумму (строка 15).

После завершения цикла суммирования мы находим общую сумму, складывая частичные суммы из каждого процесса (строка 16). Этим занимается функция редукции:

MPI_Reduce.

Напомним, что редукция — это нахождение общего итога из частичных сумм. Мы уже рассматривали редукцию в предыдущей работе.

В данной программе мы вызываем функцию редукции так:

MPI_Reduce (&S, &Z, 1, MPI_LONG_LONG, MPI_SUM, 0, MPI_COMM_WORLD);

Аргументы функции следующие:

&S — указатель на частичную сумму, которую нужно будет получить от каждого процесса;

&Z — указатель на общую сумму, которую нужно будет найти;

1 — количество данных, которое мы получаем от каждого процесса при сборе частичных сумм (в нашей программе мы получаем одно число от каждого процесса);

MPI_LONG_LONG — тип переменной, которую передаём в качестве частичной суммы (в нашей программе это длинное целое);

MPI_SUM — способ получения общей суммы (в нашей программе это операция сложения);

0 — номер главного процесса, которому сообщат общую сумму (у нас это будет процесс номер 0);

MPI_COMM_WORLD — имя коммуникатора (группы параллельных процессов), в рамках которого проводится сбор частичных сумм.

Далее главный процесс с номером ноль выводит на экран общую сумму (строки 17 и 18).

E:\Arkov-MPI\mpi5-reduce\mpi5-reduce.c

```c
#include <stdio.h>
#include <mpi.h>
#include <stdlib.h>

void main(int argc, char* argv[])
{
    int ProcessRank, CommunicatorSize;
    long long N, i, S = 0, Z;
    MPI_Init(&argc, &argv);
    N = atoll(argv[1]);
    MPI_Comm_rank(MPI_COMM_WORLD, &ProcessRank);
    MPI_Comm_size(MPI_COMM_WORLD, &CommunicatorSize);
    for (i = ProcessRank; i < N; i+= CommunicatorSize)
        S++;
    printf("S[%d] = %lld\n",ProcessRank, S);
    MPI_Reduce(&S, &Z, 1, MPI_LONG_LONG, MPI_SUM, 0, MPI_COMM_WORLD);
    if ( ProcessRank == 0 )
        printf("%lld\t%lld\n", N, Z);
    MPI_Finalize();
}
```

Рис. 7.1. Редукция

Задание. Составьте программу (рис. 7.1).

Запускаем программу на выполнение несколько раз.

Результаты правильные (рис. 7.2).

Обращаем внимание на порядок вывода сообщений на экран.

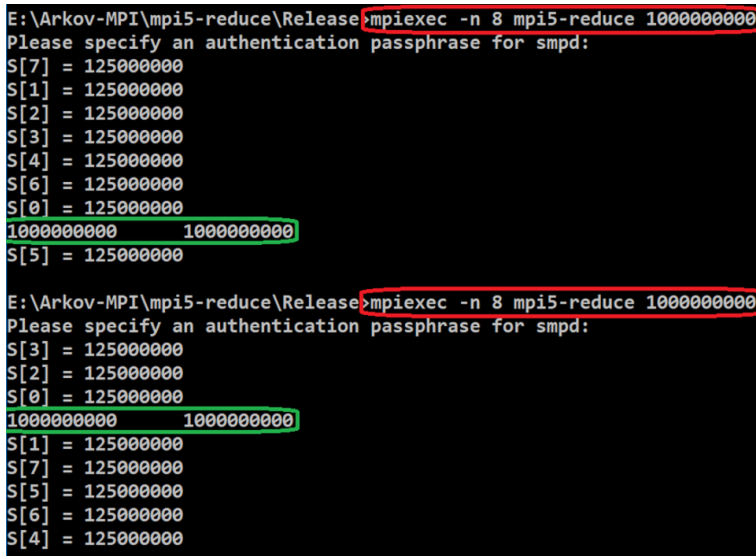


Рис. 7.2. Запуск параллельного приложения

Задание. Запустите параллельное приложение несколько раз и обратите внимание на порядок вывода сообщений на экран.

8. ВРЕМЯ ВЫПОЛНЕНИЯ

Мы сформировали задачу для параллельного приложения. Нам нужно определить время вычислений, чтобы потом найти ускорение и эффективность.

Добавим в наше программу ещё несколько строк (рис. 8.1).

Подключаем библиотеку для работы с таймером (строка 4):

#include <time. h>

Фиксируем время начала и окончания расчётов (строки 16 и 22):

t0 = clock ().

t1 = clock ().

Здесь время измеряется в отсчётах системного таймера.

Переводим время в секунды (строка 23):

T = (double) (t1 — t0) / CLOCKS_PER_SEC

Выводим результаты на экран (строка 24).

E:\Arkov-MPI\mpi6-clock\mpi6-clock.c

```c
#include <stdio.h>
#include <mpi.h>
#include <stdlib.h>
#include <time.h>

void main(int argc, char* argv[])
{
    int ProcessRank, CommunicatorSize;
    long long N, i, S = 0, Z;
    double T;
    MPI_Init(&argc, &argv);
    N = atoll(argv[1]);
    MPI_Comm_rank(MPI_COMM_WORLD, &ProcessRank);
    MPI_Comm_size(MPI_COMM_WORLD, &CommunicatorSize);
    clock_t t0, t1;
    t0 = clock();
    for (i = ProcessRank; i < N; i += CommunicatorSize)
        S++;
     MPI_Reduce(&S, &Z, 1, MPI_LONG_LONG, MPI_SUM, 0, MPI_COMM_WORLD);
    if (ProcessRank == 0)
    {
        t1 = clock();
        T = (double)(t1 - t0) / CLOCKS_PER_SEC;
        printf("%d\t%f\t%lld\n", CommunicatorSize, T, Z);
    }
    MPI_Finalize();
}
```

Рис. 8.1. Замер времени

Задание. Составьте программу (рис. 8.1).

Чтобы облегчить себе жизнь, сделаем некоторые дополнительные настройки. Но для этого понадобятся права администратора.

Нажимаем кнопку **Пуск**:

Start.

Вводим название программы, обслуживающей командное окно:

cmd.

Запускаем командное окно с правами администратора (рис. 8.2).

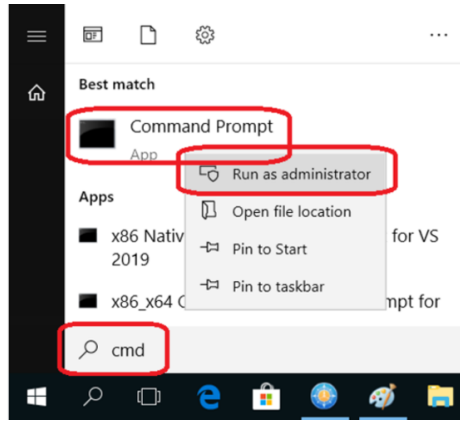


Рис. 8.2. Запуск командного окна

Задание. Запустите командное окно с правами администратора.

Чтобы не вводить пароль при каждом запуске, зарегистрируем текущего пользователя в системе MPI (рис. 8.3).

Вводим следующие команды:

smpd -install
mpiexec -remove
mpiexec -register
mpiexec -validate
smpd -status

В результате получаем сообщение о работе **smpd**.

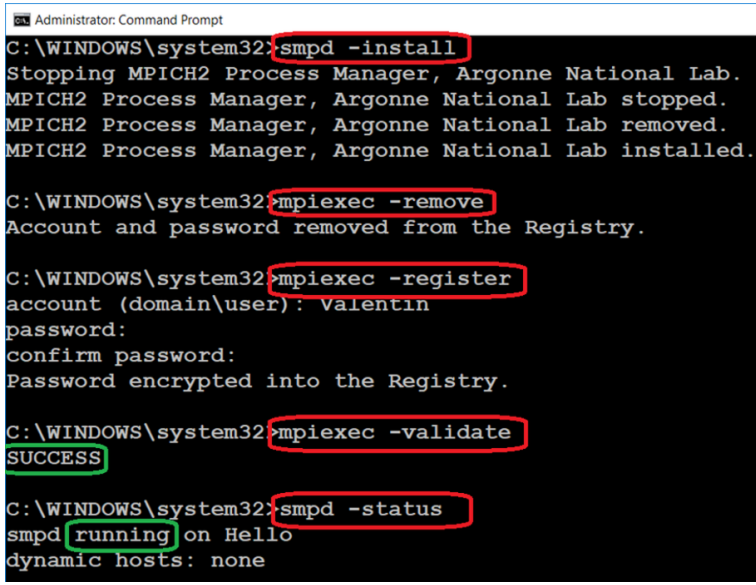


Рис. 8.3. Регистрация пользователя

Задание. Зарегистрируйте текущего пользователя в системе MPI.

Запускаем наше параллельное приложение.
На этот раз нам не требуется вводить пароль (рис. 8.4).

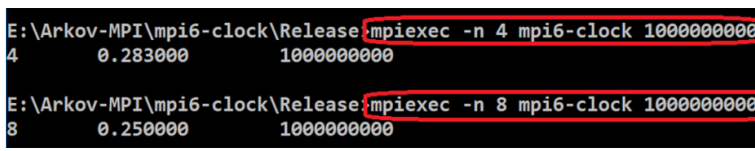


Рис. 8.4. Запуск без пароля

Задание. Запустите параллельное приложение и убедитесь в том, что ввод пароля не требуется.

9. ПАКЕТНЫЙ ЗАПУСК

Для обработки данных создадим пакетный файл (рис. 9.1).

Эту технологию мы тоже подробно рассмотрели в предыдущей работе.

Обратим внимание, что цикл нужно записывать в одну строку. При просмотре файла может показаться, что это две строчки.

На самом деле, это просто автоматический перенос при выводе содержимого файла на экран.

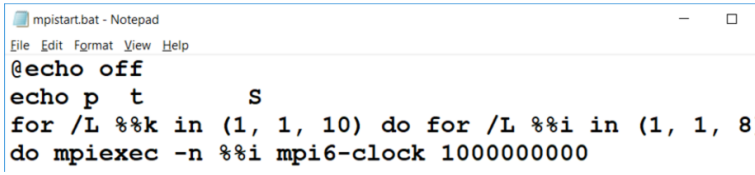


Рис. 9.1. Пакетный файл

Задание. Создайте пакетный файл (рис. 9.1).

Запускаем пакетный файл на выполнение.

Перенаправляем вывод в текстовый файл:

mpistart> m. txt.

Получаем текстовый файл (рис. 9.2).

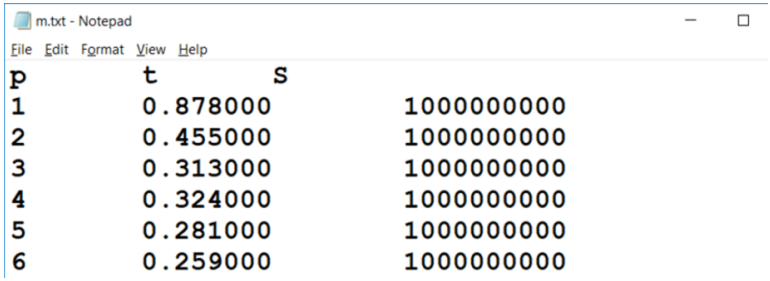


Рис. 9.2. Результаты пакетного запуска

Задание. Запустите пакетный файл и получите текстовый файл с результатами работы параллельного приложения.

Загружаем текстовый файл в **Excel**.

Этот шаг мы тоже рассмотрели в предыдущей работе.

Убеждаемся, что сумма во всех строчках одна и та же.

Значит, при расчётах ошибок не было.

Удаляем столбец сумм.

Выделяем диапазон ячеек и форматируем как таблицу с заголовками.

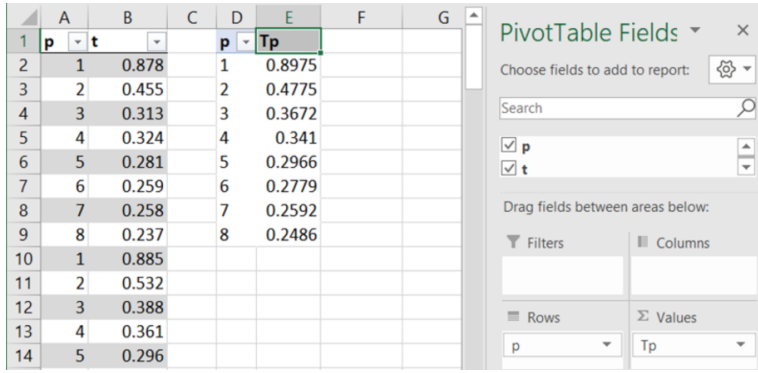
Вставляем сводную таблицу на основе наших данных.

Настраиваем сводную таблицу следующим образом:

— строки — количество процессов;

— значения — среднее время выполнения.

Удаляем итоги (рис. 9.3).



	A	B	C	D	E	F	G
1	p	t		p	Tp		
2	1	0.878		1	0.8975		
3	2	0.455		2	0.4775		
4	3	0.313		3	0.3672		
5	4	0.324		4	0.341		
6	5	0.281		5	0.2966		
7	6	0.259		6	0.2779		
8	7	0.258		7	0.2592		
9	8	0.237		8	0.2486		
10	1	0.885					
11	2	0.532					
12	3	0.388					
13	4	0.361					
14	5	0.296					

Рис. 9.3. Сводная таблица

Задание. Загрузите текстовый файл в Excel и сформируйте сводную таблицу.

Построим график времени выполнения в зависимости от количества процессов (рис. 9.4).

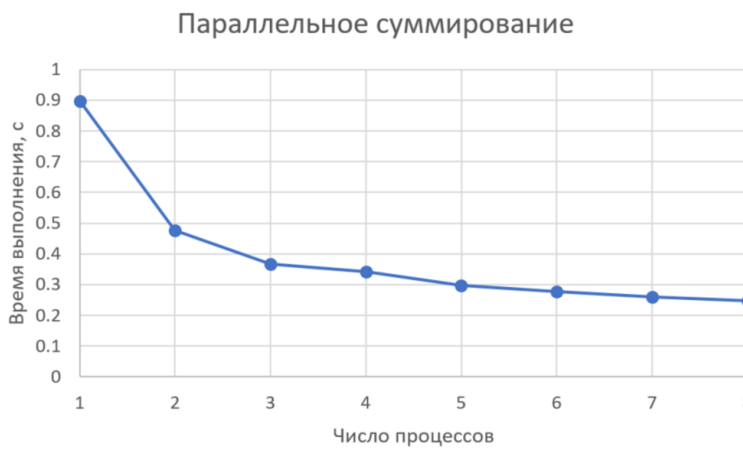


Рис. 9.4. Время суммирования

Задание. Постройте график (рис. 9.4).

Скопируем сводную таблицу в буфер обмена.

Вставим данные из буфера как значения.

Добавим вычисление ускорения и эффективности (рис. 9.5).

Будем считать, что число процессоров равно числу процессов. Конечно, это очень приблизительный анализ. И не совсем корректный. Мы про это ещё поговорим.

p	Tp	Sp	Ep
1	0.8975	1.0000	100%
2	0.4775	1.8796	94%
3	0.3672	2.4442	81%
4	0.3410	2.6320	66%
5	0.2966	3.0260	61%
6	0.2779	3.2296	54%
7	0.2592	3.4626	49%
8	0.2486	3.6102	45%

Рис. 9.5. Ускорение и эффективность

Задание. Определите показатели эффективности распараллеливания.

Построим график зависимости ускорения вычислений от числа процессов (рис. 9.6).

Можно видеть, что ускорение на восьми процессах не поднимается даже до четырёх. И это на четырёх ядрах с гипертредингом.

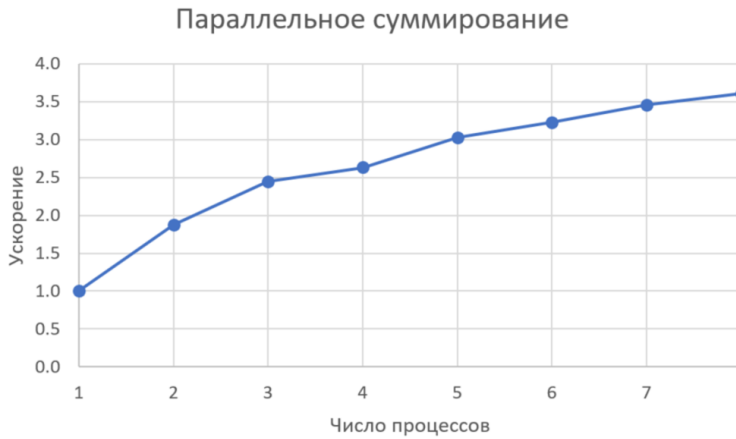


Рис. 9.6. Ускорение вычислений

Задание. Постройте график (рис. 9.6) и оцените максимальное ускорение.

10. ПАРАЛЛЕЛЬНОЕ ИНТЕГРИРОВАНИЕ

Мы отработали все этапы исследования. Переходим к решению задачи численного интегрирования. В нашу программу придётся добавить всего несколько строк (рис. 10.1).

Обсудим основные шаги, которые выполняются в нашей программе. Лучше разобраться в программе и понять её — на следующем этапе исследований вам придётся вносить в неё небольшие изменения.

Счётчик цикла и число итераций — переменные типа длинное целое (строка 9).

В первом варианте программы мы используем одинарную точность. Объявляем переменные типа **float** (строка 10).

Определяем номер процесса в группе (строка 13).

Определяем общее количество параллельных процессов в группе (строка 14).

Фиксируем время начала работы (строка 15).

Находим шаг по «икс» — ширину прямоугольника (строка 16).

Начинаем цикл с раздачей заданий методом чередования (строка 17).

Аргумент подынтегральной функции в нашем варианте меняется с выбранным шагом, начиная от **–20** (строка 19).

Вычисляем значение функции в середине прямоугольника (строка 20).

После завершения всех циклов вызываем редукцию (строка 22). Обратим внимание на указание типа переменных при обмене сообщениями:

MPI_FLOAT.

Этот тип переменных для частичной и общей суммы.

После завершения редукции нулевой процесс подводит итоги (строка 24). Остальные процессы завершают работу.

Умножаем общую сумму на ширину основания прямоугольников (строка 26).

Фиксируем время завершения работы (строка 27).

Пересчитываем количество сигналов системного таймера в секунды (строка 28).

Выводим на экран результаты, разделяя их символом табуляции (строка 29):

— количество параллельных процессов;
— количество прямоугольников;
— время вычислений в секундах;
— оценка значения определённого интеграла.

E:\Arkov-MPI\mpi7-integral\mpi7-integral.c

```c
#include <stdio.h>
#include <mpi.h>
#include <stdlib.h>
#include <time.h>

void main(int argc, char* argv[])
{
    int ProcessRank, CommunicatorSize;
    long long N, i;
    float T, dx, x, Z, S = 0.0;
    MPI_Init(&argc, &argv);
    N = atoll(argv[1]);
    MPI_Comm_rank(MPI_COMM_WORLD, &ProcessRank);
    MPI_Comm_size(MPI_COMM_WORLD, &CommunicatorSize);
    clock_t t0 = clock();
    dx = 40.0 / N;
    for (i = ProcessRank; i < N; i += CommunicatorSize)
    {
        x = -20.0 + (i + 0.5) * dx;
        S = S + 0.05 * x * x * x + 0.2 * x * x - 10.0 * x + 200.0;
    }
    MPI_Reduce(&S, &Z, 1, MPI_FLOAT, MPI_SUM, 0, MPI_COMM_WORLD);
    MPI_Finalize();
    if (ProcessRank == 0)
    {
        Z = Z * dx;
        clock_t t1 = clock();
        T = (float)(t1 - t0) / CLOCKS_PER_SEC;
        printf("%d\t%lld\t%.10f\t%.10f\n", CommunicatorSize, N, T, Z);
    }
}
```

Рис. 10.1. Параллельное интегрирование

Задание. Составьте программу (рис. 10.1).

Компилируем программу.

Запускаем исполняемый файл и убеждаемся в его работоспособности.

Запускаем параллельное приложение с помощью утилиты **mpiexec**. Всё работает (рис. 10.2).

```
E:\Arkov-MPI\mpi7-integral\Release>mpi7-integral 1000
1       1000    0.0010000000    9066.6669921875

E:\Arkov-MPI\mpi7-integral\Release>mpiexec -n 8 mpi7-integral 100000000
8       100000000       0.1959999949    8813.6757812500

E:\Arkov-MPI\mpi7-integral\Release>mpiexec -n 8 mpi7-integral 10000000
8       10000000        0.0250000004    9071.1015625000

E:\Arkov-MPI\mpi7-integral\Release>mpiexec -n 8 mpi7-integral 1000000
8       1000000 0.0250000004    9066.8496093750
```

Рис. 10.2. Запуски программы

Задание. Скомпилируйте программу параллельного численного интегрирования и убедитесь в её работоспособности.

Следующий шаг исследований — пакетный запуск по десять прогонов для каждого набора параметров и анализ в электронной таблице. Для анализа используйте сводные таблицы. Результаты исследования представьте в виде графиков. При необходимости используйте логарифмический масштаб. Все эти шаги подробно описаны в предыдущем пособии.

Задание. Создайте пакетный файл и исследуйте результаты работы программы в пакете Excel. Постройте следующие графики:

— оценка значения интеграла в зависимости от числа прямоугольников и числа параллельных процессов;

— ошибка оценки значения интеграла в зависимости от числа прямоугольников и числа параллельных процессов;

— ускорение и эффективность в зависимости от числа параллельных процессов.

Следующий этап исследования — учёт типа переменных. Изменим тип переменных для расчётов на вещественные числа с двойной точностью (строка 10):

double.

При этом нужно указать тип переменных для системы обмена сообщениями (строка 22):

MPI_DOUBLE

Задание. Повторите исследование и постройте описанные выше графики.

11. Задания повышенной сложности

В этом разделе описаны дополнительные, необязательные задания, которые выполняются по выбору пациента. То есть по желанию студента. Который действительно хочет что-то новое узнать и чему-то научиться. Обычно таких желающих набирается не слишком много. Остальным достаточно простого результата типа «сдать и забыть».

11.1. Гипертрединг

Если осталось время и силы, можно проделать ещё одно небольшое исследование в отношении гипертрединга.

Технология **Intel HyperThreading** сводится к тому, что каждое физическое ядро процессора выглядит как два логических процессора для операционной системы.

Будем исходить из следующего предположения:

— ядро 0 — логические процессоры 0 и 1;

— ядро 1 — логические процессоры 2 и 3;

— ядро 2 — логические процессоры 4 и 5

и так далее.

Как мы уже обсуждали в предыдущем пособии, гипертрединг — это НЕ два полноценных ядра вместо одного физического. Речь идёт о двух «входных» устройствах, принимающих «задания» от двух параллельных потоков. Эта технология помогает обеспечить более полную загрузку вычислительных элементов ОДНОГО ядра. Поэтому здесь нельзя ожидать ускорения работы в два раза. Ускорение будет гораздо меньше. И в этом можно убедиться, если честно выполнить все задания.

Попробуем загрузить работой каждое ядро без учёта гипертрединга. Проведём два эксперимента с привязкой процессов к ядрам — с помощью функции **SetThreadAffinityMask**.

Первый опыт — привязка процессов ко всем логическим процессорам.

— процесс 0 — логический процесс 0;

— процесс 1 — логический процессор 1;

— процесс 2 — логический процессор 2

и так далее.

Второй опыт — привязка процессов к физическим ядрам без загрузки второго логического ядра:

— процесс 0 — логический процесс 0;

— процесс 1 — логический процессор 2;

— процесс 2 — логический процессор 4

и так далее.

Для процессора с четырьмя ядрами в обоих опытах запускаем по четыре параллельных процесса. Следим за 100% загрузкой вычислительных элементов в **Диспетчере задач.**

При таком подходе можно будет оценить фактическое ускорение за счёт гипертрединга — четыре логических процессора против четырёх физических ядер.

Задание. Определите фактическое ускорение, которое можно достичь при численном интегрировании за счёт гипертрединга.

11.2. Распределённые вычисления

Мы рассмотрели, как запустить параллельное приложение MPI на ОДНОМ многоядерном процессоре.

Главное предназначение, ради чего была разработана технология MPI — это распределённые вычисления. Термин «распределённые» (distributed) означает, что вычисления по одной задаче выполняются на нескольких компьютерах. Это МНОГОПРОЦЕССОРНЫЕ системы, так называемые КЛАСТЕРЫ.

Чтобы познакомиться с технологией распределённых вычислений, нужно иметь доступ к нескольким компьютерам, соединённым локальной сетью. Это могут быть персональные компьютеры в дисплейном классе. Это могут быть несколько домашних компьютеров — настольных или ноутбуков, подключённых к од-

ному маршрутизатору. Это может быть суперкомпьютер, к которому предоставлен доступ.

Поскольку параллельные процессы общаются, то есть обмениваются сообщениями по сети, здесь требуется быстрая локальная сеть. Соединение через интернет может существенно замедлить работу.

В этом случае для опытов нужно будет подобрать такой размер задачи, чтобы расчёты занимали не десятки секунд, а десятки минут или часов. Решение реальных задач потребует несколько часов или дней параллельных вычислений.

Начнём с домашних опытов. Если дома есть хотя бы два компьютера, можно провести наш эксперимент.

Для работы службы smpd на всех компьютерах должен быть зарегистрирован пользователь с одним и тем же логином и с одинаковым паролем. При этом пароль не должен быть пустым. Пусть в пароле будут хотя бы какие-нибудь символы. Можно завести специального пользователя на время экспериментов, а потом его удалить.

На каждом компьютере нужно установить библиотеку MPI. Если на втором компьютере не будет запускаться компилятор, можно скопировать все необходимые файлы в сетевую папку с общим доступом. Для удобства работы лучше расположить папку в корневом каталоге рабочего диска.

Нам потребуются следующие файлы (рис. 11.1):

— утилита **mpiexec. ex**e из каталога **bin**;

— менеджер процессов **smpd. exe** из каталога **bin**;

— все библиотеки **mpi*.dll** из каталога **system32**;

— скомпилированное параллельное приложение, которое предстоит запускать.

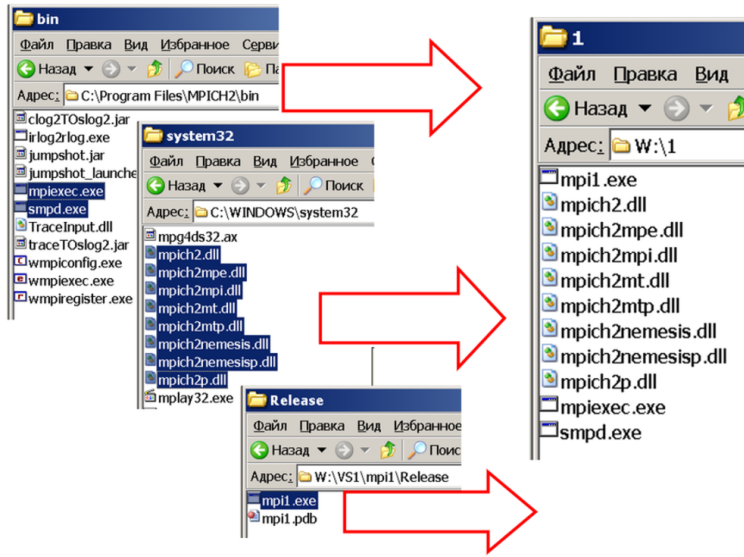


Рис. 11.1. Копирование файлов

Задание. Скопируйте перечисленные файлы в одну папку.

Папка, в которую мы скопировали все перечисленные файлы, нужно сделать «разделяемой», то есть доступной по сети для других компьютеров. Жаргонное название: «расшарить» — от английского слова SHARE.

Щёлкаем правой кнопкой по папке, выбираем **Свойства** (**Properties**) и **Общий доступ** (**Sharing**). Затем нужно убедиться, что с другого (удалённого) компьютера можно зайти на локальный и увидеть содержимое сетевой папки.

Задание. Настройте общий доступ к папке.

Следующий шаг — определить IP-адреса компьютеров. Обычно для этого достаточно запустить командное окно и вызвать утилиту:

ipconfig.

Нас интересует сетевой адрес компьютера:

IPv4 Address.

В следующей строчке видим маску подсети:

Subnet Mask (рис. 11.2).

Возможно, эти слова кому-нибудь напомнят стандартную задачу из ЕГЭ по информатике. Это для тех, кому «посчастливилось» сдавать такой экзамен или помогать друзьям и знакомым готовиться к нему.

Следует учесть ДИНАМИЧЕСКОЕ назначение сетевых адресов. Обычно выход в интернет при наличии нескольких компьютеров из дома производится с использованием сетевого маршрутизатора, или роутера — от английского ROUTE — «маршрут». При каждом включении или перезагрузке роутера он автоматически назначает подключённым к нему компьютерам сетевые адреса. Устройства с беспроводным подключением по WiFi тоже получают свои адреса. Каждый раз эти адреса меняются. Это и есть динамическое назначение адресов, то есть назначение в процессе работы.

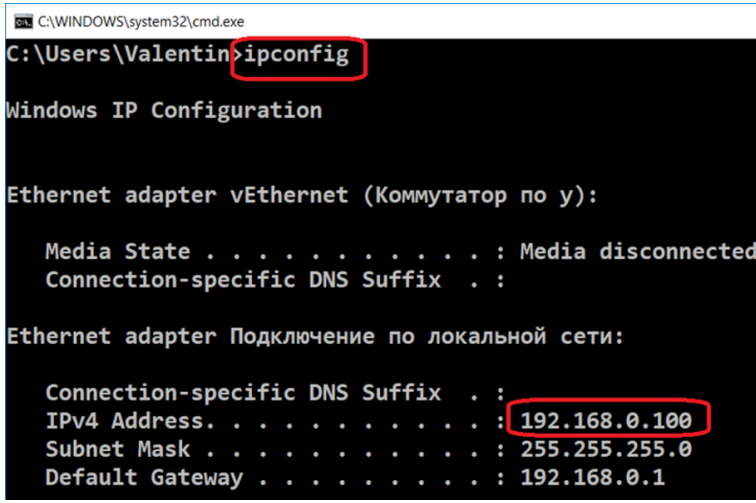


Рис. 11.2. Сетевой адрес локального компьютера

Чтобы определить сетевой адрес, придётся подойти к каждому компьютеру. На них могут быть установлены разные операционные системы (рис. 11.3).

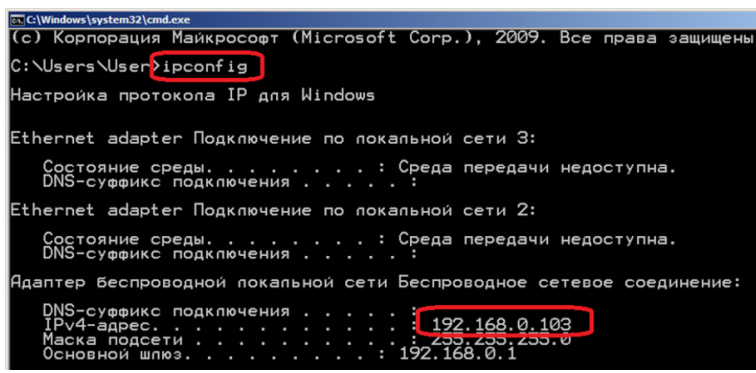


Рис. 11.3. Сетевой адрес удалённого компьютера.

Задание. Определите сетевые адреса всех компьютеров, на которых вы будете запускать параллельное приложение.

Следующий шаг — убедиться в наличии связи между компьютерами по локальной сети.

Для этого используется команда операционной системы **PING**.

Формат команды простой — в качестве параметра достаточно указать сетевой адрес второго компьютера (рис. 11.4).

Название команды, скорее всего, намекает на игру в настольный теннис под названием «пинг-понг». Это имитация звука ударов по шарику. При проверке связи локальный компьютер отправляет запрос — «пинг!». Удалённый компьютер присылает свой ответ — «понг!»

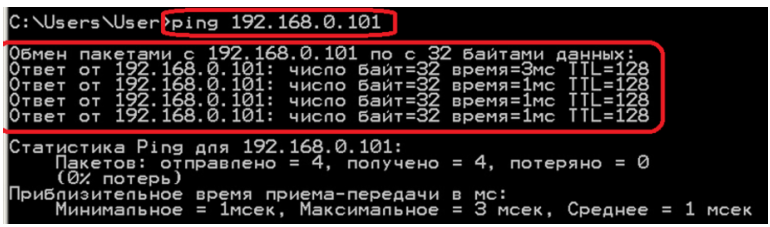


Рис. 11.4. Проверка связи

Задание. Проверьте наличие связи командой PING.

На рис. 11.5. приводится пример команды для запуска двух процессов на двух машинах. В параметрах команды указан исполняемый файл на сетевом диске:

mpiexec. exe -hosts 2 192.168.0.100 2 192.168.0.101 2 \ \\192.168.0.100\1\mpi1.exe

Результаты работы выводятся в командном окне на локальном компьютере, даже при выводе из процессов на удалённом компьютере.

```
C:\1>mpiexec.exe -hosts 2 192.168.0.100 2 192.168.0.101 2 \\192.168.0.100\1\mpi1
.exe
Hello world from process 0 of 4
Hello world from process 2 of 4
Hello world from process 3 of 4
Hello world from process 1 of 4
```

Рис. 11.5. Распределённый запуск

Задание. Запустите параллельное приложение в распределённом режиме на нескольких машинах. Обратите внимание на загрузку процессоров.

ЗАКЛЮЧЕНИЕ

Мы рассмотрели технологию параллельных вычислений с использованием библиотеки MPI. Сами программы были не слишком сложными.

Самое интересное в данном исследовании — это анализ результатов. Оценка ускорения вычислений.

К запатентованным технологиям и зарегистрированным торговым маркам следует относиться с уважением. По крайней мере, их надо знать. Хотя бы названия. Но с пониманием.

Проводя многочисленные опыты, мы убедились, что гипертрединг только называется ГИПЕР, то есть «очень-очень много». На самом деле речь идёт о ДВУХ потоках, которые выполняются на одном ядре. Причём последовательно. Так что мы лишний раз убедились, что фактическое ускорение зависит, прежде всего, от числа физических ядер.

Задание. Рассмотрите исходный текст программы в каталоге **examples** и попробуйте узнать стандартные конструкции, которые мы освоили в данной работе.

ЛИТЕРАТУРА

1. Воеводин В. В., Воеводин Вл. В. Параллельные вычисления. — СПб.: БХВ-Петербург, 2002. — 608 с.

2. Гергель В. П. Высокопроизводительные вычисления для многопроцессорных многоядерных систем. — М.: МГУ, 2010. — 544 с.

3. Основы суперкомпьютерных технологий / Р.К.Газизов, В.О.Лукащук, С.Ю.Лукащук, А.В.Юлдашев. — Уфа: УГАТУ, 2008. — 267 с.

4. Немнюгин С. А., Стесик О. Д. Параллельное программирование для многопроцессорных вычислительных систем. — СПб.: БХВ-Петербург, 2002. — 400 с.

5. Эндрюс Г. Р. Основы многопоточного, параллельного и распределенного программирования. — М.: Вильямс, 2003.– 512 с.

6. Воеводин В. В. Вычислительная математика и структура алгоритмов. — М.: МГУ, 2006. — 112 с.

Режим доступа:

http://parallel.ru/info/parallel/voevodin/

7. Воеводин Вл. В., Жуматий С. А. Вычислительное дело и кластерные системы. — М.: МГУ, 2007. — 150 с.

Режим доступа:

http://parallel.ru/info/parallel/cluster/

8. Антонов А. С. Параллельное программирование с использованием технологии OpenMP: Учебное пособие. — М.: МГУ, 2009. — 77 с.

Режим доступа:

http://parallel.ru/info/parallel/openmp/

9. Антонов А. С. Технологии параллельного программирования MPI и OpenMP. — М.: МГУ, 2012. — 344 с.

10. Арьков В. Ю. Организация параллельных потоков. Часть

1: Лабораторный практикум. — [б. м.]: Издательские решения, 2020. — 46 с.

Режим доступа:

https://ridero.ru/books/parallelnye_potoki_windows/

11. Арьков В. Ю. Организация параллельных потоков. Часть 2: Учебное пособие. — [б. м.]: Издательские решения, 2020. — 124 с.

Режим доступа:

https://ridero.ru/books/organizaciya_parallelnykh_potokov_chast_2/

12. Арьков В. Ю. Анализ распределения в электронных таблицах: Учебное пособие. — [б. м.]: Издательские решения, 2019. — 158 с.

Режим доступа:

https://ridero.ru/books/analiz_raspredeleniya_v_excel/

13. Арьков В. Ю. Бизнес-аналитика. Сводные таблицы. Часть 1: Учебное пособие. — [б. м.]: Издательские решения, 2020. — 180 с.

Режим доступа:

https://ridero.ru/books/biznes-analitika_v_excel_svodnye_tablicy/

14. MPI: A Message-Passing Interface Standard. Version 3.1. Technical report. — Message Passing Interface Forum, 2015.

Режим доступа:

https://www.mpi-forum.org/docs/mpi-3.1/mpi31-report.pdf

15. Видеоролик: MPI. Установка библиотеки MPI.

Режим доступа:

https://youtu.be/NxGU-PgGaFA

16. Видеоролик: MPI. Hello World.

Режим доступа:

https://youtu.be/b5MAKdt8L_E

17. Видеоролик: Ввод числа через параметр командной строки. Argc-Argv-Atoi.

https://youtu.be/k7taS1XB2c0

18. Видеоролик: MPI. Оценка числа Пи.

Режим доступа:
https://youtu.be/2U_34mWjTgU

ОБРАТНАЯ СВЯЗЬ

Обратная связь — вещь полезная в любом деле.

Автор не гарантирует, что в книге не будет ошибок, пробелов и неточностей. Читатель может что-нибудь такое обнаружить. А, возможно, даже появятся предложения и пожелания.

Всё это можно излить в своём отзыве. На странице книги в магазине Ридеро есть возможность оставить свой отзыв. Это внизу страницы.

ОГЛАВЛЕНИЕ

Введение ... 3
0. Организация работы ... 4
  0.1. Отчёт ... 4
  0.2. Варианты заданий ... 5
  0.3. Среда разработки ... 6
1. Технология MPI ... 7
2. Установка библиотеки MPI ... 8
3. Программа «Всем привет!» ... 19
  3.1. Настройка каталога Include ... 20
  3.2. Настройка каталога lib ... 25
  3.3. Подключение библиотеки mpi.lib ... 28
  3.4. Запуск программы ... 30
4. Номер процесса ... 37
5. Параметры командной строки ... 40
6. Параллельное суммирование ... 49
7. Редукция ... 55
8. Время выполнения ... 59
9. Пакетный запуск ... 64
10. Параллельное интегрирование ... 69
Заключение ... 81
Литература ... 82
Обратная связь ... 85

Валентин Юльевич Арьков

Организация параллельных процессов

Учебное пособие

Создано в интеллектуальной издательской системе Ridero

Арьков Валентин Юльевич — доктор технических наук, профессор кафедры автоматизированных систем управления Уфимского государственного авиационного технического университета. Опыт преподавания в вузе более 30 лет.

В данной работе мы познакомимся с организацией высокопроизводительных вычислений на уровне параллельных процессов. При этом большую задачу приходится разделить на несколько частей, а выполнение каждой части работы поручить отдельному процессу.
Первые шаги мы рассмотрим подробно, а потом отпустим читателя в свободное плавание.

ISBN 978-5-4498-6204-4

Rideró

Rideró.ru — издай книгу бесплатно!